

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 9: GREEDY TECHNIQUE

Abdullah Bal, PhD

Objectives



Recap

Dynamic Programming
Change making
Coin Collecting
Floyd's Algorithm



Greedy Technique



Prim's Algorithm Dijkstra's Algorithm



Huffman Coding

Change-Making Problem

- Let's say we have unlimited coins with the denominations

$$d_1 = 25, d_2 = 10, d_3 = 5, \text{ and } d_4 = 1.$$

- How would we give change with coins of these denominations of, say, 48 cents?
- If you came up with the answer 1 quarter, 2 dimes, and 3 pennies, you followed a logical strategy of making a sequence of best choices among the currently available alternatives.
- Indeed, in the first step, you could have given one coin of any of the four denominations.

Change-Making Problem

- “Greedy” thinking leads to giving one quarter because it reduces the remaining amount the most, namely, to 23 cents.
- In the second step, you had the same coins at your disposal, but you could not give a quarter, because it would have violated the problem’s constraints.
- So your best selection in this step was one dime, reducing the remaining amount to 13 cents.
- Giving one more dime left you with 3 cents to be given with three pennies.

Change-Making Problem

- Is this solution to the instance of the change-making problem optimal?
- Yes, it is.
- In fact, one can prove that the greedy algorithm yields an optimal solution for every positive integer amount with these coin denominations.
- At the same time, it is easy to give an example of coin denominations that **do not yield an optimal solution** for some amounts

$$d_1 = 25, d_2 = 10, d_3 = 3 \text{ and } n = 30.$$

Greedy Technique

- This change-making problem solution technique is called *greedy*.
- Computer scientists consider it a general design technique despite the fact that **it is applicable to optimization problems only**.
- The greedy approach suggests constructing **a solution through a sequence of steps**, each expanding a partially constructed solution obtained so far, until a complete solution to the problem is reached.
- **On each step**—and this is the central point of this technique—the choice made must be:
 - **feasible**, i.e., it has to satisfy the problem's constraints
 - **locally optimal**, i.e., it has to be the best local choice among all feasible choices available on that step
 - **irrevocable**, i.e., once made, it cannot be changed on subsequent steps of the algorithm
- For some problems, yields an optimal solution for every instance.
- For most, does not but can **be useful for fast approximations**.

Applications of the Greedy Strategy

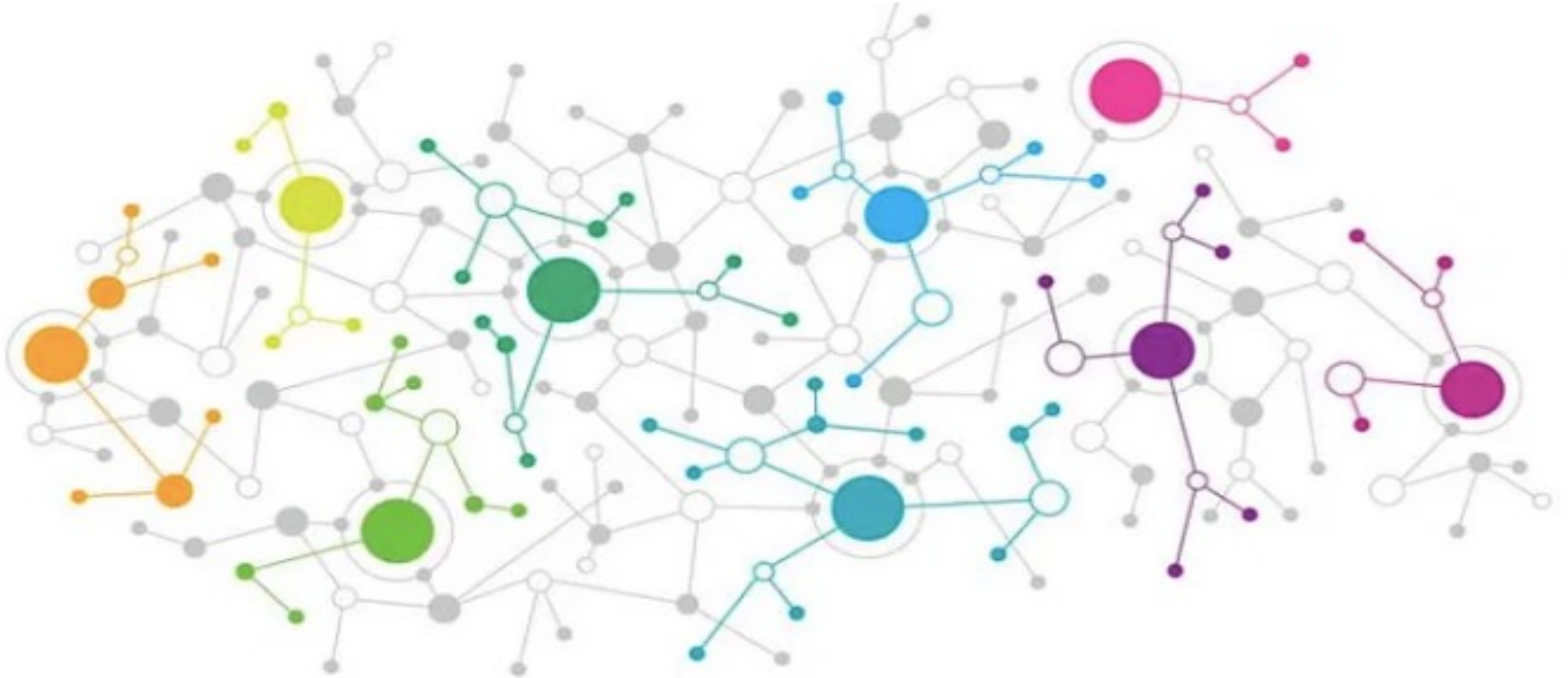
Optimal solutions:

- change making for “normal” coin denominations
- **minimum spanning tree (MST)**
- **single-source shortest paths**
- simple scheduling problems
- **Huffman codes**

Approximations:

- traveling salesman problem (TSP)
- knapsack problem
- other combinatorial optimization problems

Prim's Algorithm (Minimum Spanning Tree)



Minimum Spanning Tree (MST)

Spanning tree of a connected graph G :

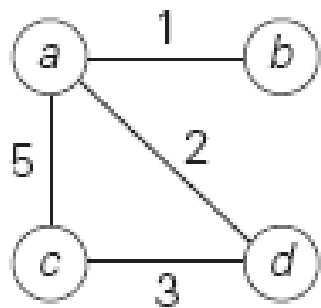
- a connected **acyclic** subgraph of G that includes all of G 's vertices

Minimum spanning tree of a weighted, connected graph G :

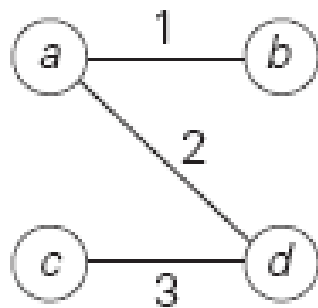
- a spanning tree of G of **minimum total weight**

Minimum Spanning Tree (MST) Problem

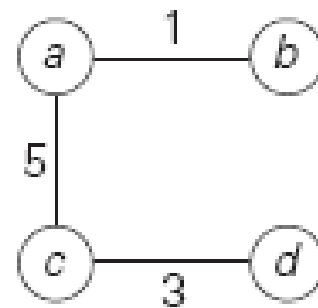
- Graph and its spanning trees, with T_1 being the minimum spanning tree.



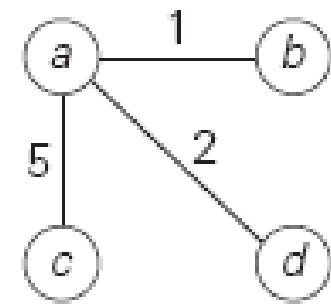
graph



$w(T_1) = 6$



$w(T_2) = 9$



$w(T_3) = 8$

Minimum Spanning Tree (MST)

DEFINITION A *spanning tree* of an undirected connected graph is its **connected acyclic subgraph** (i.e., a tree) that contains **all the vertices** of the graph.

- If such a graph has weights assigned to its edges, a ***minimum spanning tree*** is its spanning tree of the smallest weight, where the ***weight*** of a tree is defined as the sum of the weights on all its edges.

Prim's MST algorithm

- If we were to try constructing a minimum spanning tree by **exhaustive search**, we would face two serious obstacles.
 - First, the number of spanning trees **grows exponentially with the graph size**.
 - Second, generating all spanning trees for a given graph is not easy; in fact, it is more difficult than finding a *minimum* spanning tree for a weighted graph by using one of several efficient algorithms available for this problem.
- In this section, we outline ***Prim's algorithm***, which goes back to at least 1957.

Prim's MST algorithm

- Prim's algorithm constructs a minimum spanning tree through a sequence of expanding subtrees.
- The initial subtree in such a sequence consists of a single vertex **selected arbitrarily** from the set V of the graph's vertices.
- On each iteration, the algorithm **expands the current tree in the greedy manner** by simply attaching to it the nearest vertex not in that tree.
- By the nearest vertex, we mean a vertex not in the tree connected to a vertex in the tree **by an edge of the smallest weight**.
- The algorithm **stops after all the graph's vertices have been included** in the tree being constructed.
- Since the algorithm expands a tree by exactly one vertex on each of its iterations, the total number of such iterations is $n - 1$, where n is the number of vertices in the graph.

Prim's MST algorithm

- We can provide such information by attaching two labels to a vertex: **the name of the nearest tree vertex** and **the length (the weight) of the corresponding edge**.
- Vertices that are not adjacent to any of the tree vertices can be given the **∞ label** indicating their “infinite” distance to the tree vertices and a **null label** for the name of the nearest tree vertex.
- With such labels, finding the next vertex to be added to the current tree becomes a simple task of finding a vertex with the smallest distance label in the set $V - V_T$.

Prim's MST algorithm

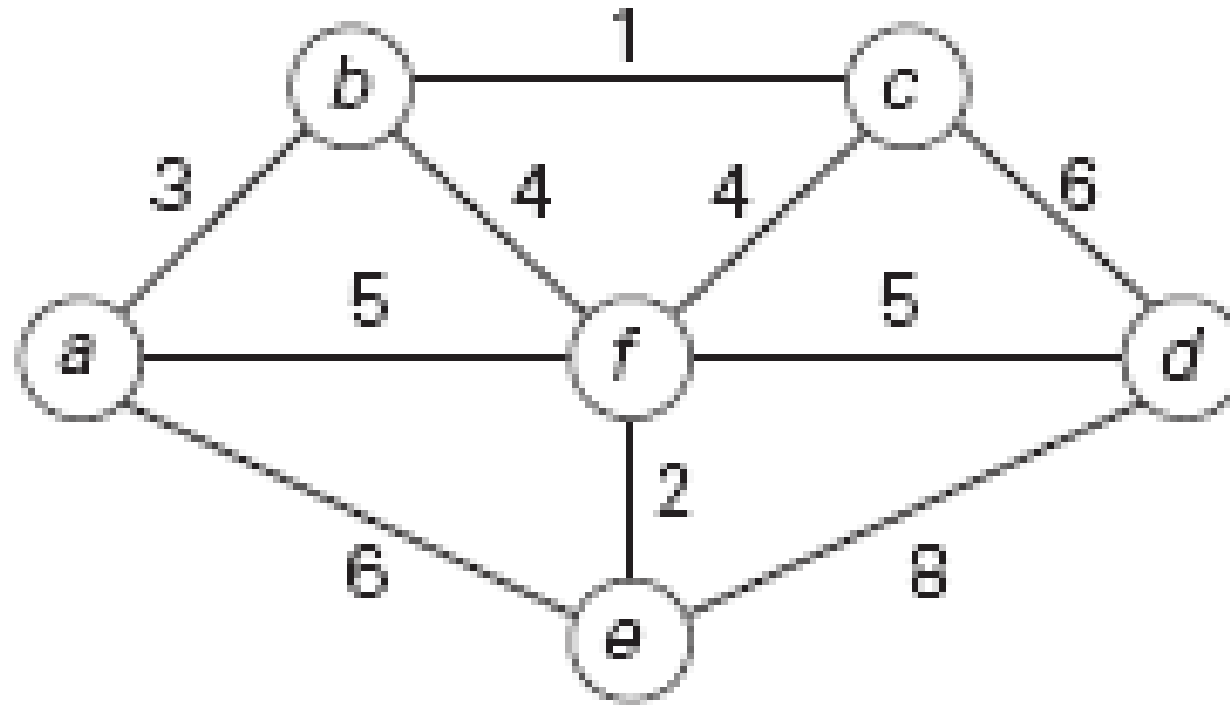
- Start with tree T_1 consisting of one (any) vertex and “grow” tree one vertex at a time to produce MST through a series of expanding subtrees

$$T_1, T_2, \dots, T_n$$

- On each iteration, construct T_{i+1} from T_i by adding vertex not in T_i that is closest to those already in T_i (this is a “greedy” step!)
- Stop when all vertices are included

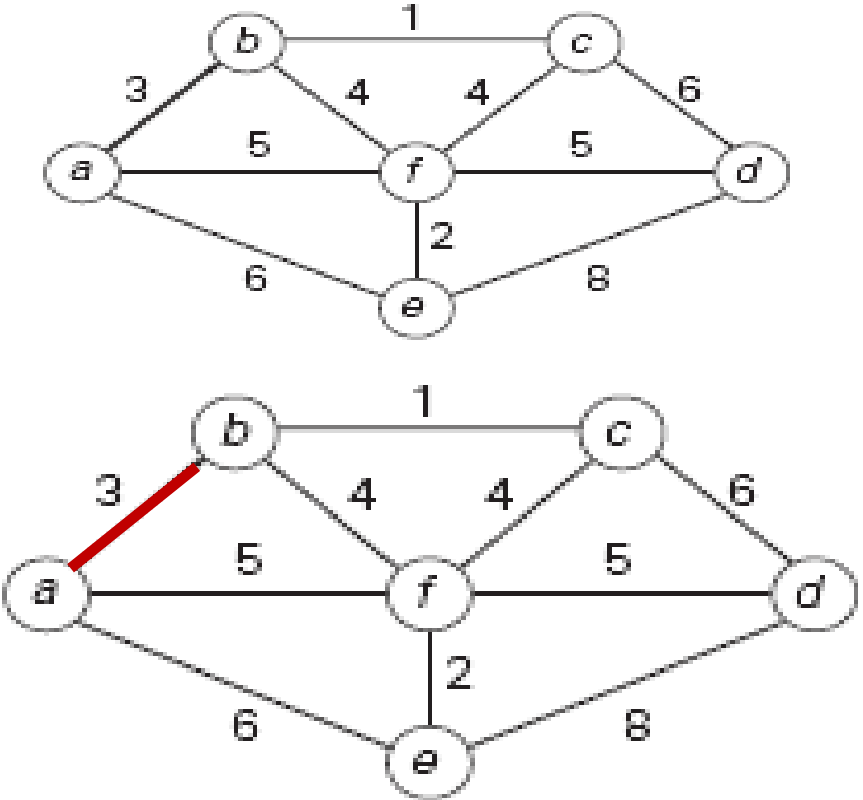
Example 1: Prim's MST algorithm

- Construct the minimum spanning tree for following graph:



Example 1: Prim's MST algorithm

Step 1

Tree vertices	Remaining vertices	Illustration
$a(-, -)$	$b(a, 3)$ $c(-, \infty)$ $d(-, \infty)$ $e(a, 6)$ $f(a, 5)$	
Selected Vertex $b(a, 3)$		

Example 1: Prim's MST algorithm

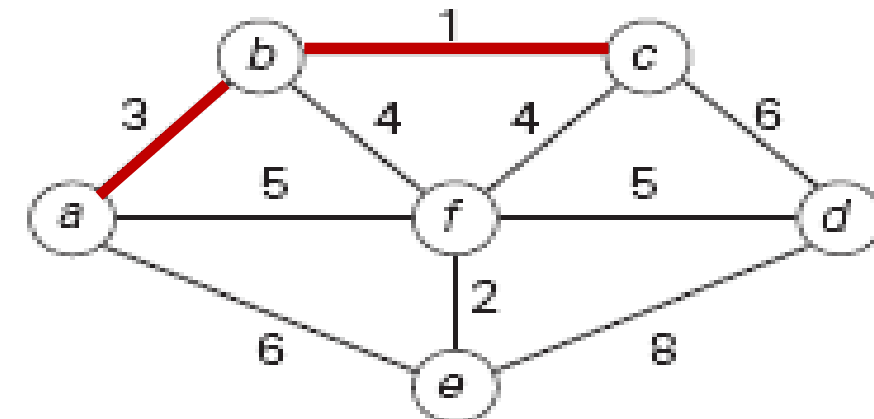
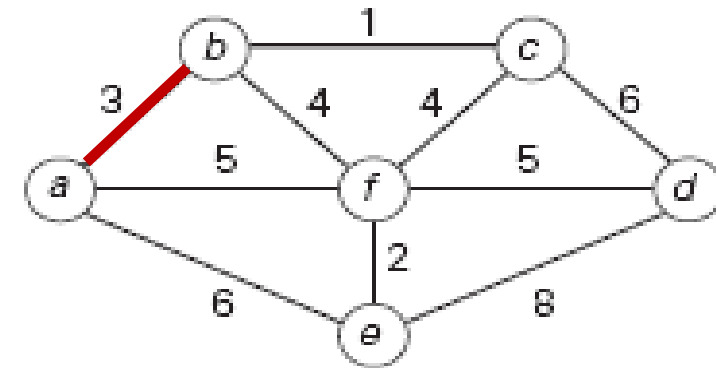
Step 2

Tree vertices

Remaining vertices

Illustration

$c(b, 1)$ $d(-, \infty)$ $e(a, 6)$
 $f(b, 4)$



Selected Vertex
 $c(b, 1)$

Example 1: Prim's MST algorithm

Step 3

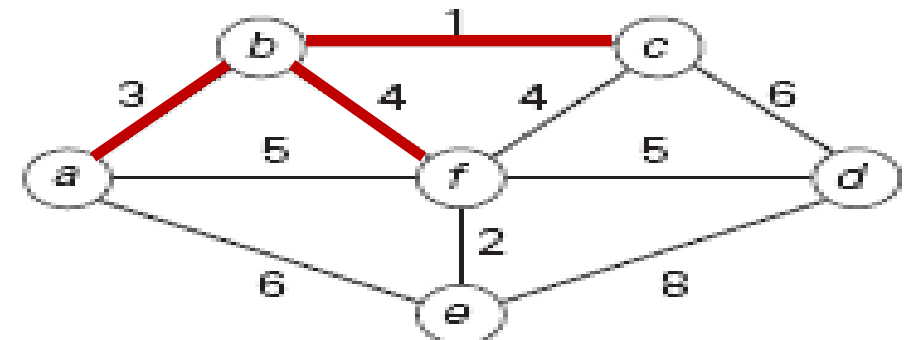
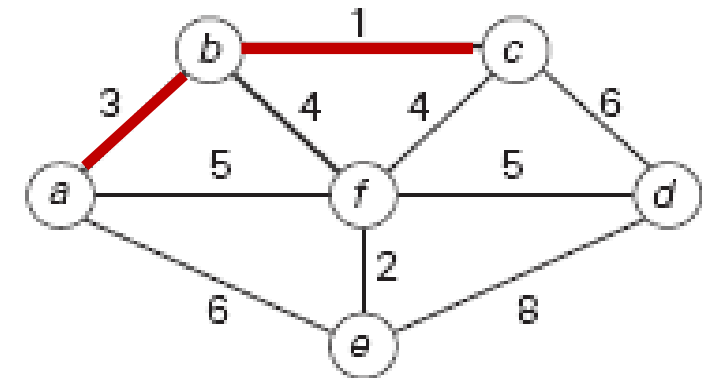
Tree vertices

Remaining vertices

Illustration

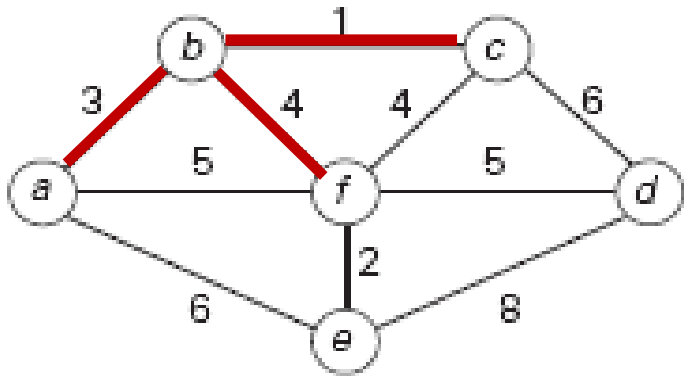
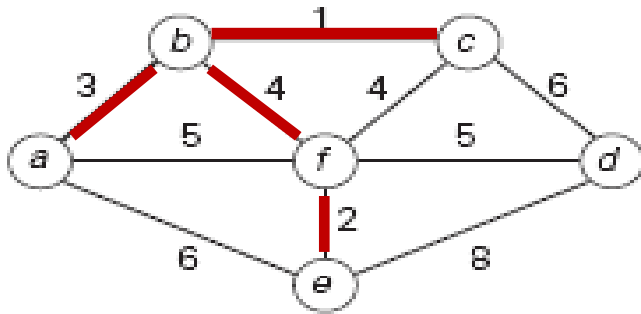
$d(c, 6)$ $e(a, 6)$ $f(b, 4)$

Selected Vertex
 $f(b, 4)$



Example 1: Prim's MST algorithm

Step 4

Tree vertices	Remaining vertices	Illustration
	$d(f, 5) \quad e(f, 2)$	
Selected Vertex $e(f, 2)$		

Example 1: Prim's MST algorithm

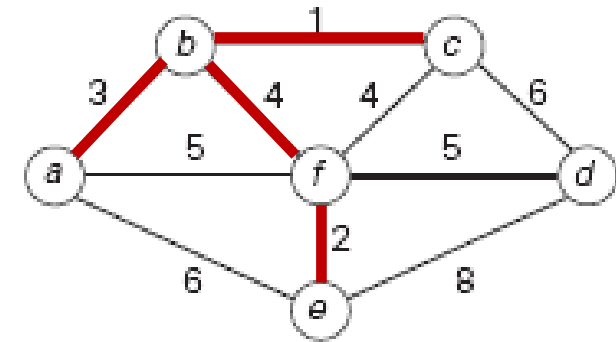
Step 5

Tree vertices

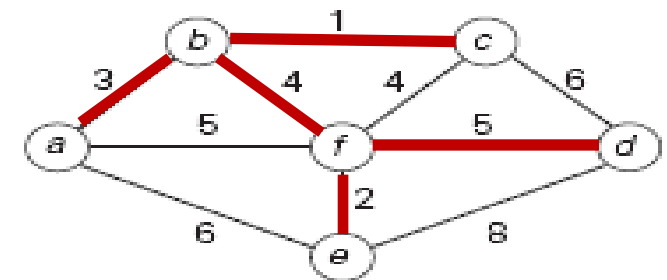
Remaining vertices

Illustration

$d(f, 5)$

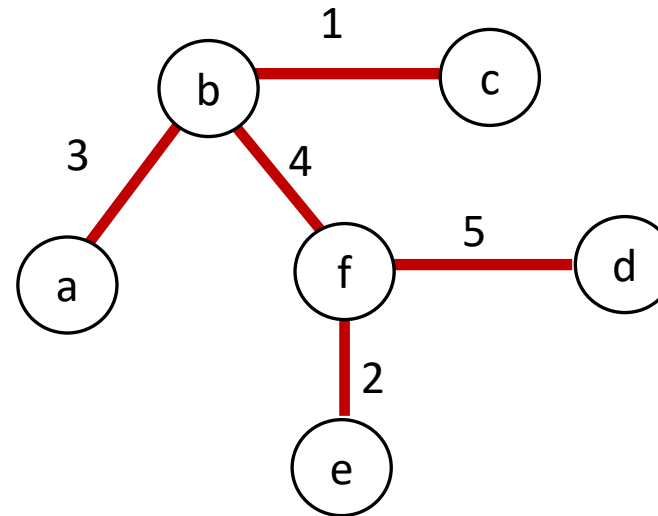


Selected Vertex
 $d(f, 5)$



Example 1: Prim's MST algorithm

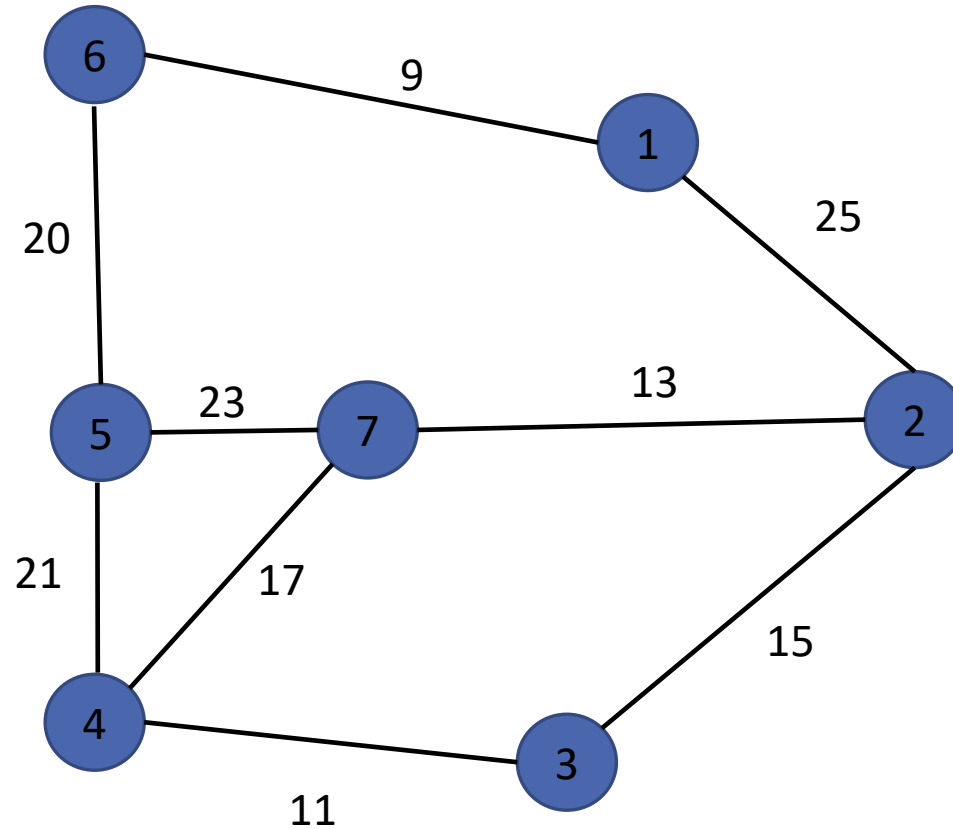
- Constructed MST:



$$W(T) = 3 + 1 + 4 + 5 + 2 = 15$$

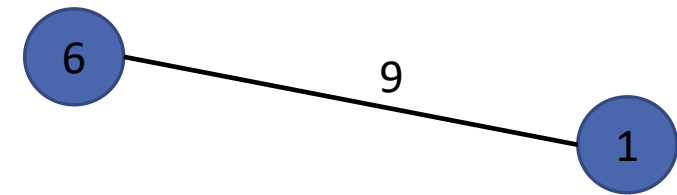
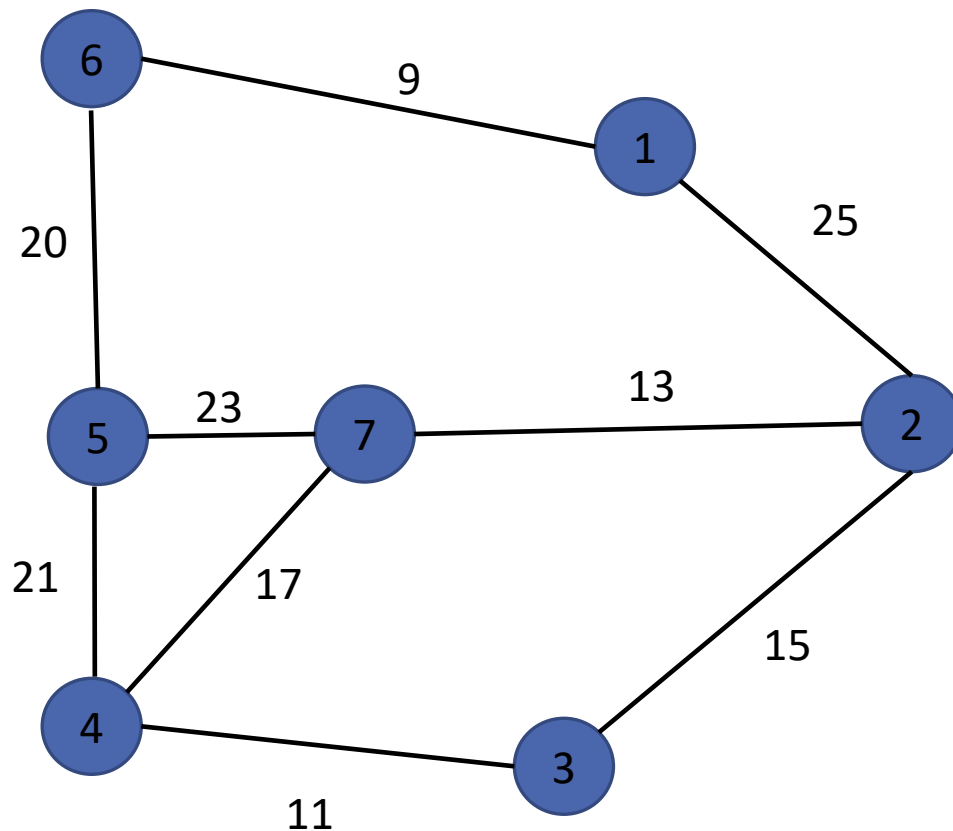
Example 2: Prim's MST algorithm

- Construct the minimum spanning tree for following graph:



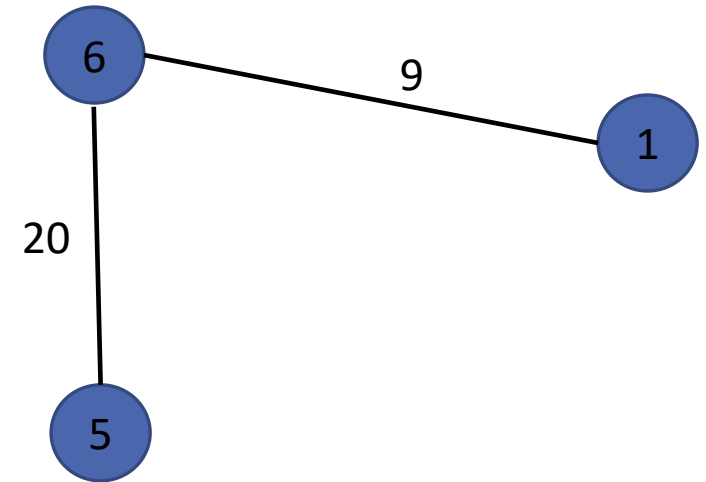
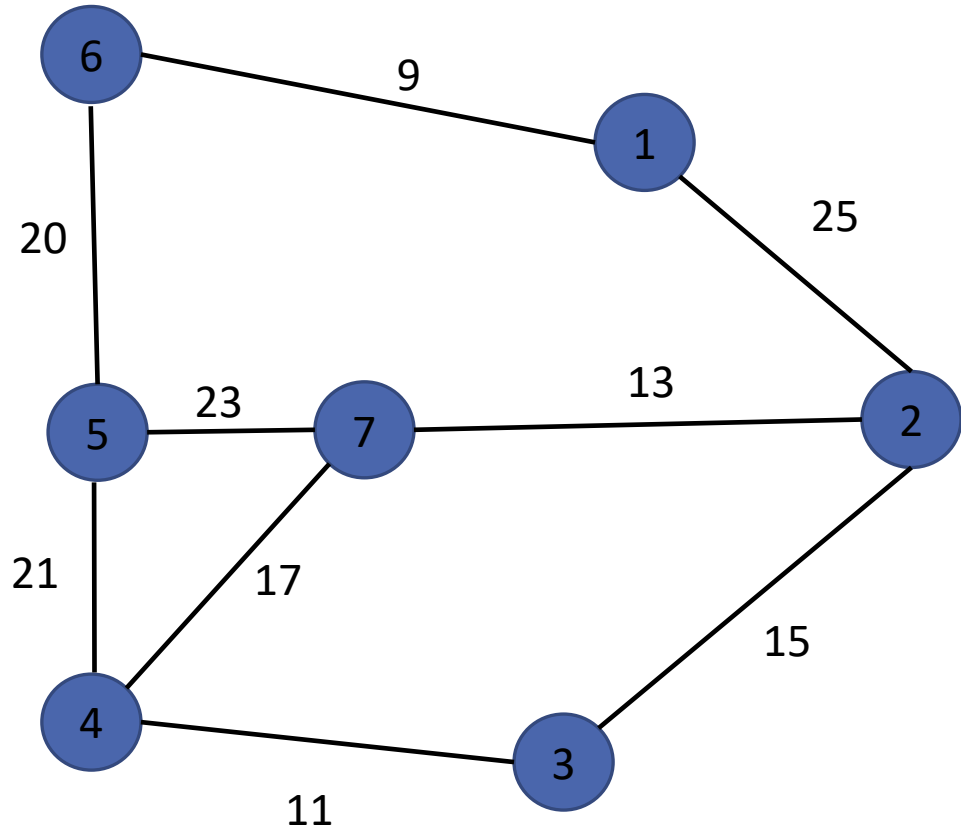
Example 2: Prim's MST algorithm

Step 1



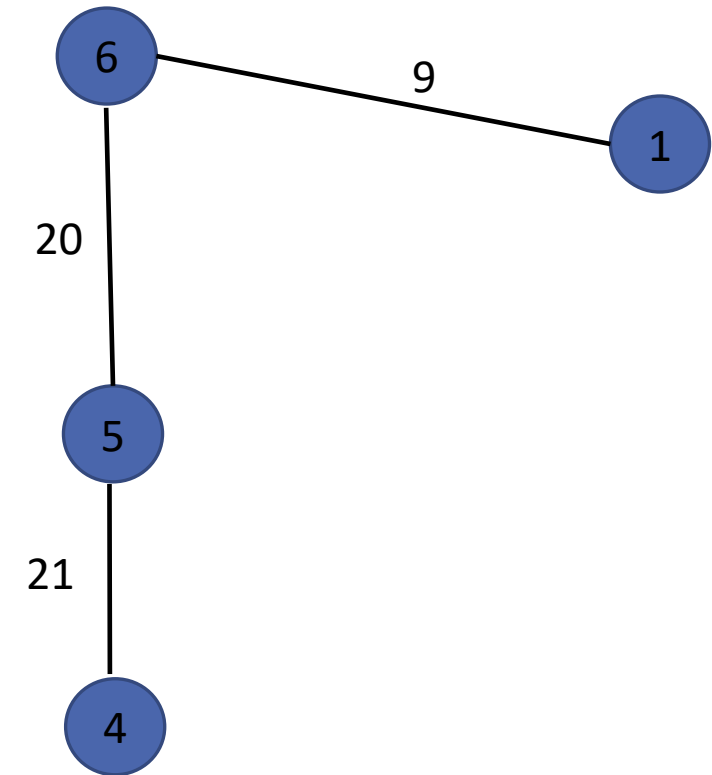
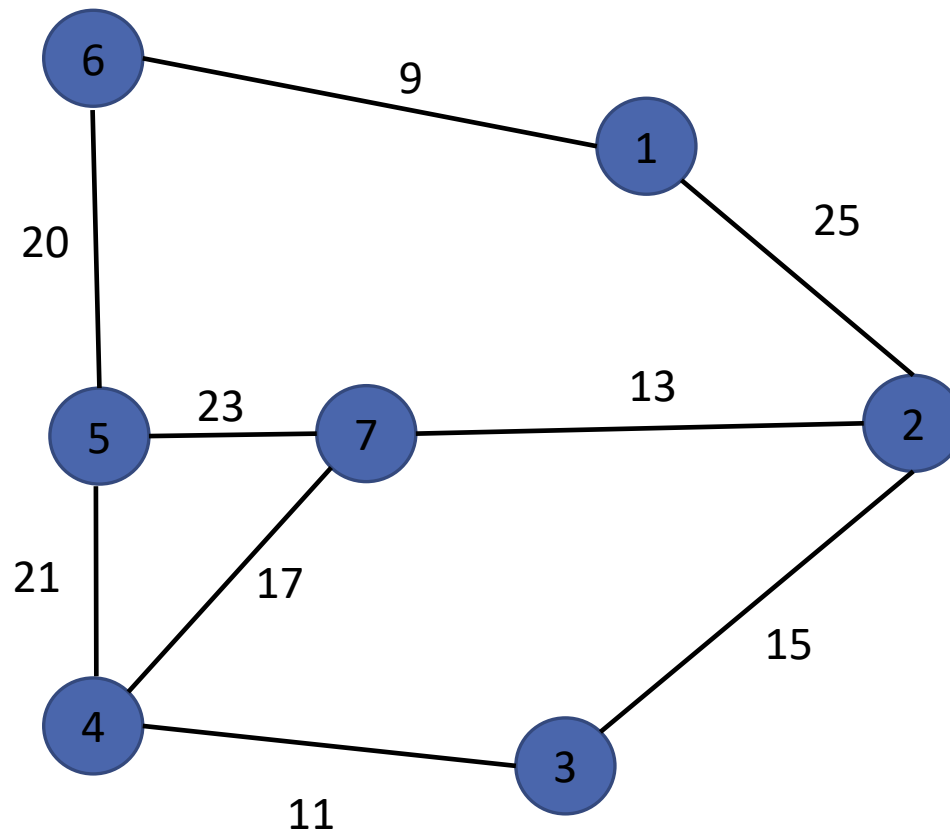
Example 2: Prim's MST algorithm

Step 2



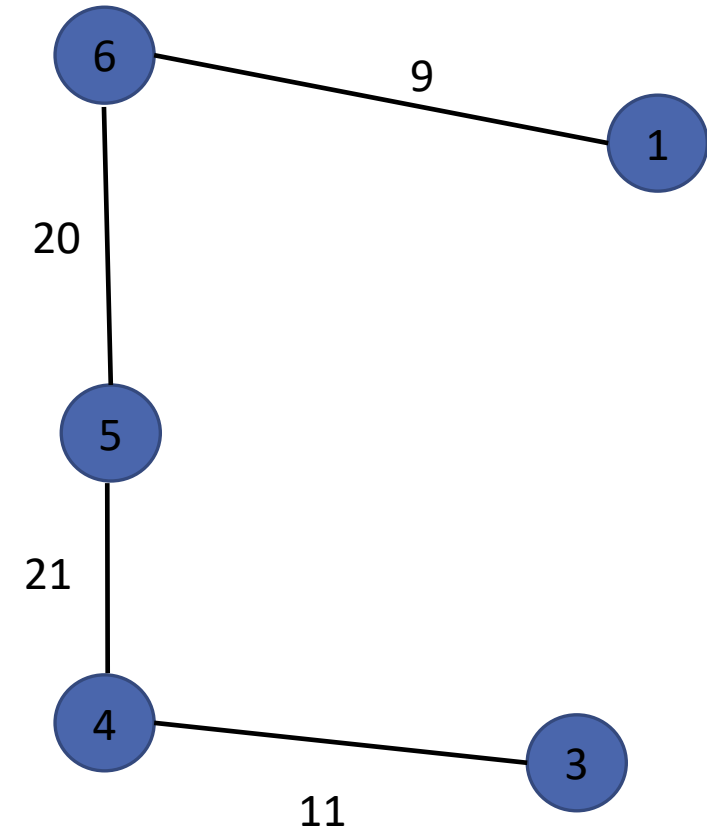
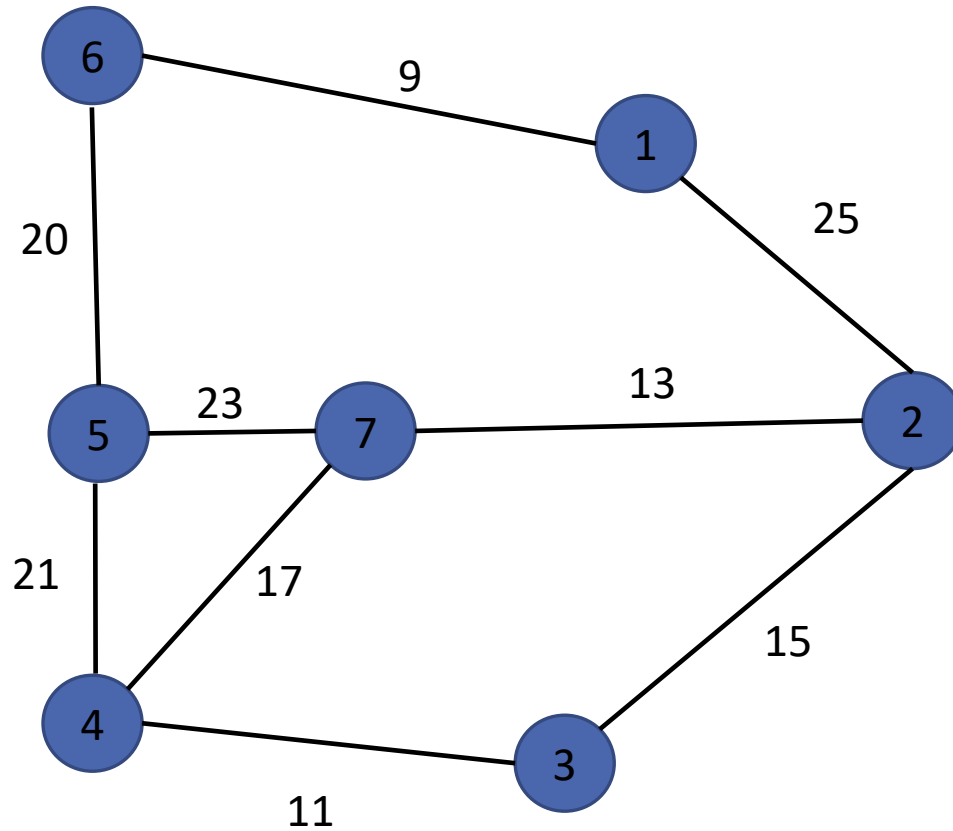
Example 2: Prim's MST algorithm

Step 3



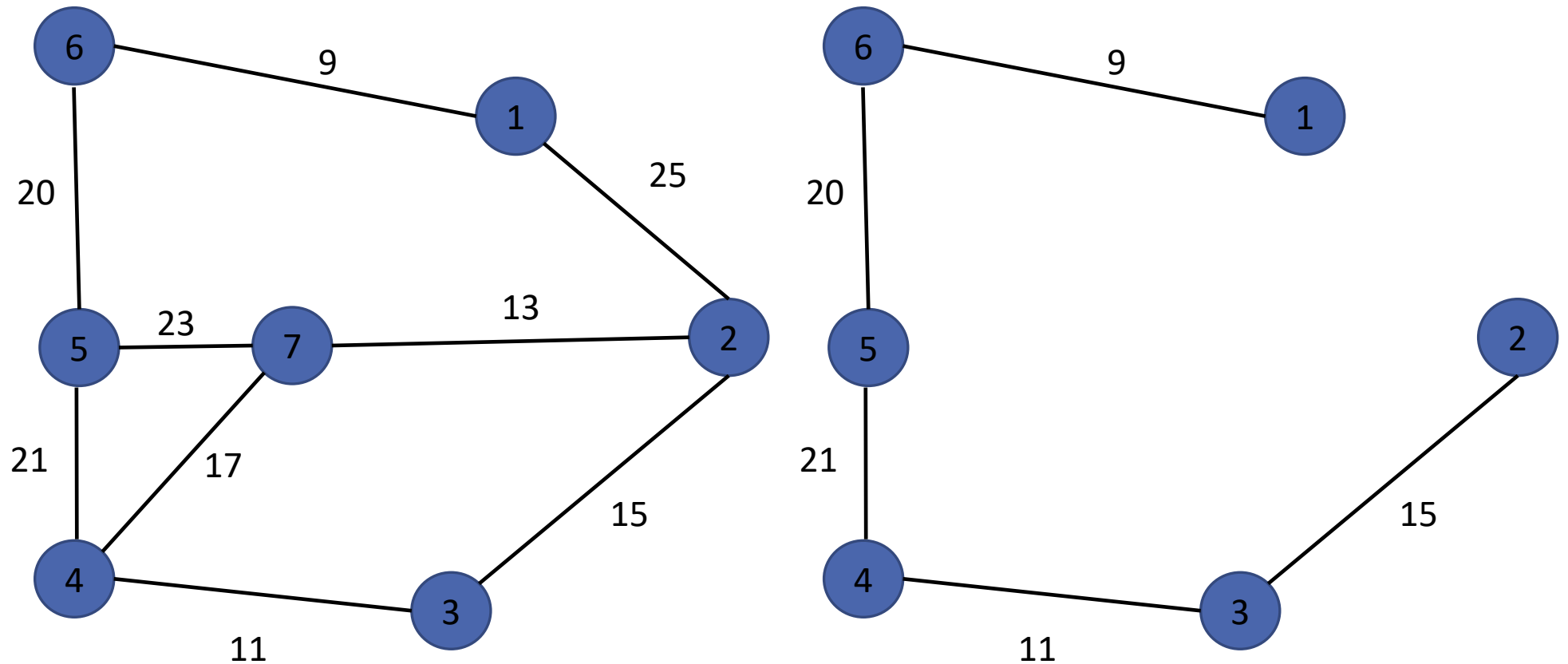
Example 2: Prim's MST algorithm

Step 4



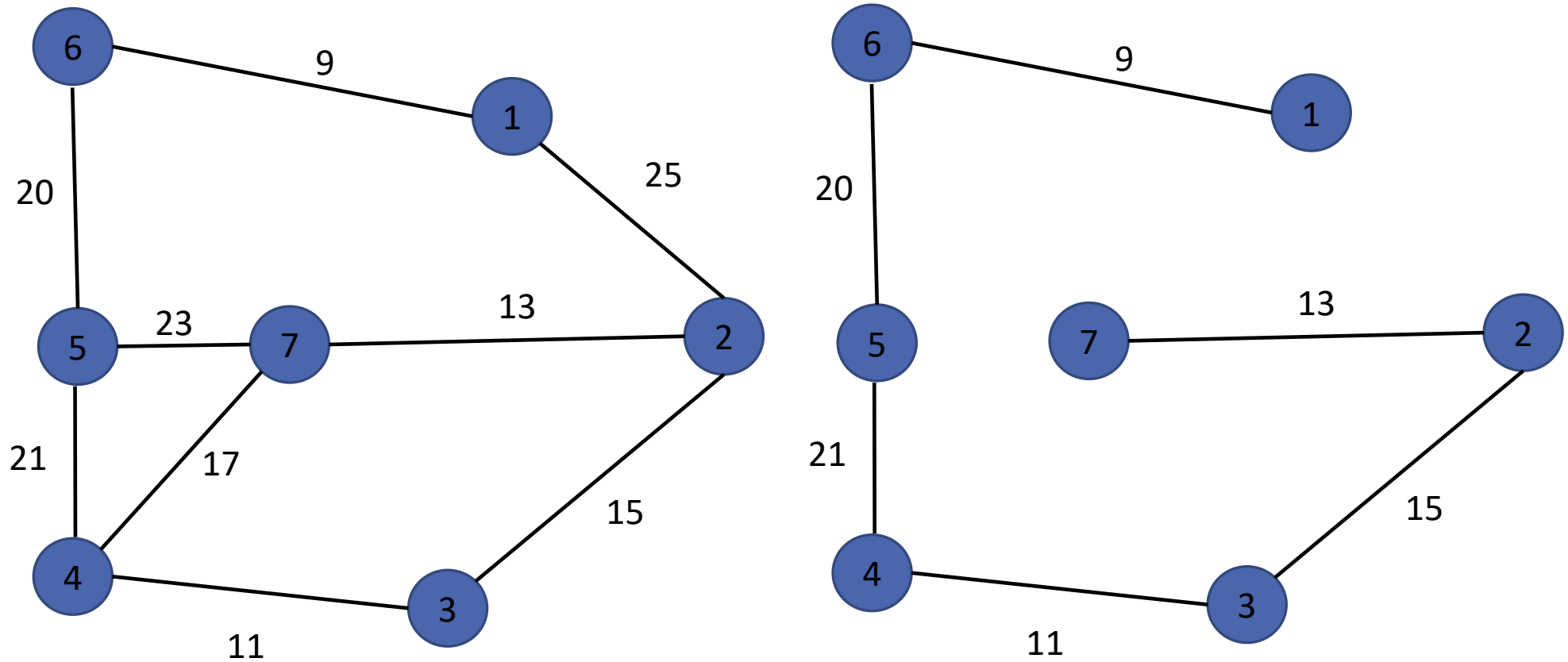
Example 2: Prim's MST algorithm

Step 5



Example 2: Prim's MST algorithm

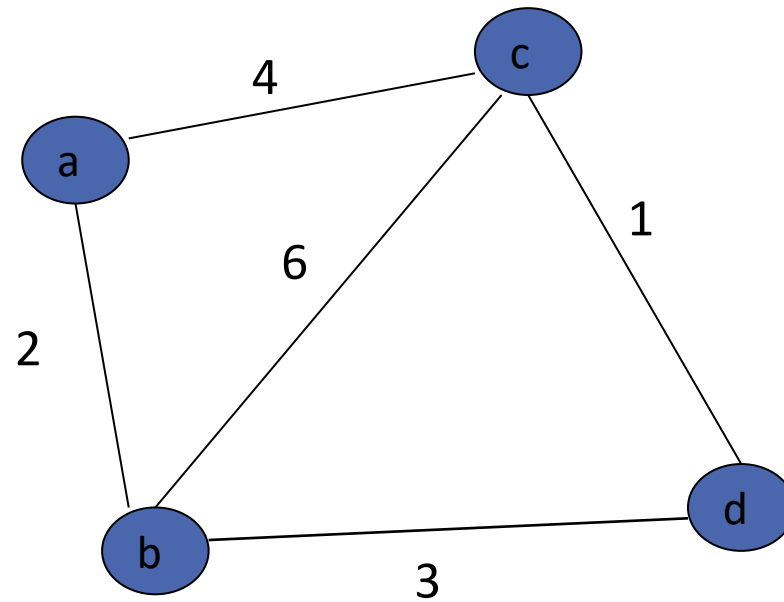
Step 6



$$W(T) = 9 + 20 + 21 + 11 + 15 + 13 = 89$$

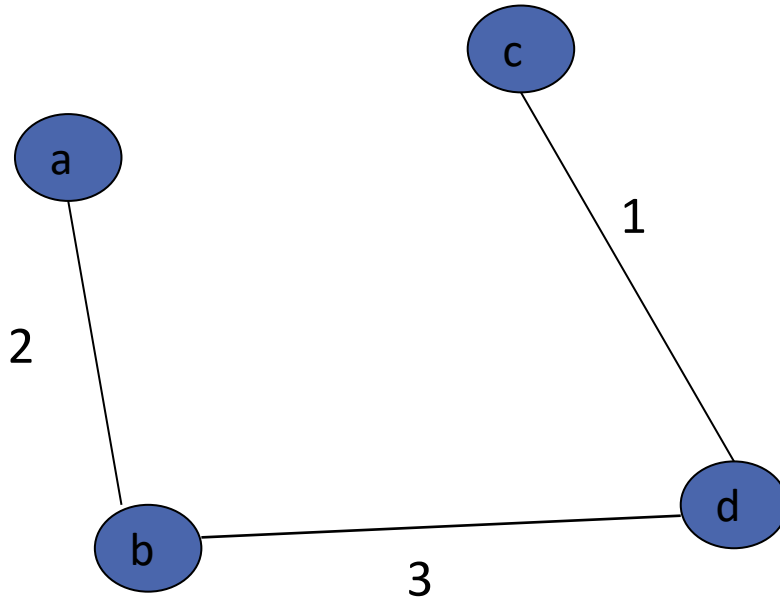
Example 3: Prim's MST algorithm

- Construct the minimum spanning tree for following graph:



Example 3: Prim's MST algorithm

- Constructed MST:



$$W(T) = 1 + 2 + 3 = 6$$

Notes about Prim's algorithm

- Needs priority queue for locating closest suitable vertex
- Efficiency
 - $O(n^2)$ for weight matrix representation of graph and array implementation of priority queue
 - $O(m \log n)$ for adjacency list representation of graph with n vertices and m edges and min-heap implementation of priority queue

D

Dijkstra's algorithm

S

— : 14 min 53 sec

- - - : 15 min 30 sec

... : 15 min 48 sec

Single-source Shortest-paths Problem

- **Single-source shortest-paths problem**: for a given vertex called the **source** in a weighted connected graph, **find shortest paths to all its other vertices**.
- The single-source shortest-paths problem asks for a family of paths, each leading from the source to a different vertex in the graph, though some paths may, of course, have edges in common.
- The obvious but probably most widely used applications are **transportation planning** and **packet routing in communication networks**, including the **Internet**.
- Multitudes of less obvious applications include finding shortest paths in **social networks**, **speech recognition**, **document formatting**, and **robotics**.
- In the world of entertainment, one can mention **pathfinding in video games** and finding best solutions to puzzles using their **state-space graphs**.

Shortest paths – Dijkstra's algorithm

- There are several well-known algorithms for finding shortest paths, including **Floyd's algorithm** for the more general **all-pairs shortest-paths problem**.
- We consider the best-known algorithm for the **single-source shortest-paths problem**, called ***Dijkstra's algorithm***.
- This algorithm is applicable to **undirected and directed graphs with nonnegative weights only**.
- Since in most applications this condition is satisfied, the limitation has **not impaired** the popularity of Dijkstra's algorithm.
- Edsger W. Dijkstra (1930–2002), a noted Dutch pioneer of the science and industry of computing, discovered this algorithm in the **mid-1950s**.
- Dijkstra's algorithm finds **the shortest paths to a graph's vertices in order of their distance from a given source**.

Shortest paths – Dijkstra's algorithm

- It finds the shortest path from the source to a vertex nearest to it, then to a second nearest, and so on.
- In general, before its i^{th} iteration commences, the algorithm has already identified the shortest paths to $i - 1$ other vertices nearest to the source.
- These vertices, the source, and the edges of the shortest paths leading to them from the source form a subtree T_i of the given graph.
- Since all the edge weights are nonnegative, the next vertex nearest to the source can be found among the vertices adjacent to the vertices of T_i .
- The set of vertices adjacent to the vertices in T_i can be referred to as “fringe vertices”; they are the candidates from which Dijkstra's algorithm selects the next vertex nearest to the source.
- To identify the i^{th} nearest vertex, the algorithm computes, for every fringe vertex u , the sum of the distance to the nearest tree vertex v (given by the weight of the edge (v, u)) and the length d_v of the shortest path from the source to v and then selects the vertex with the smallest such sum.

Shortest paths – Dijkstra's algorithm

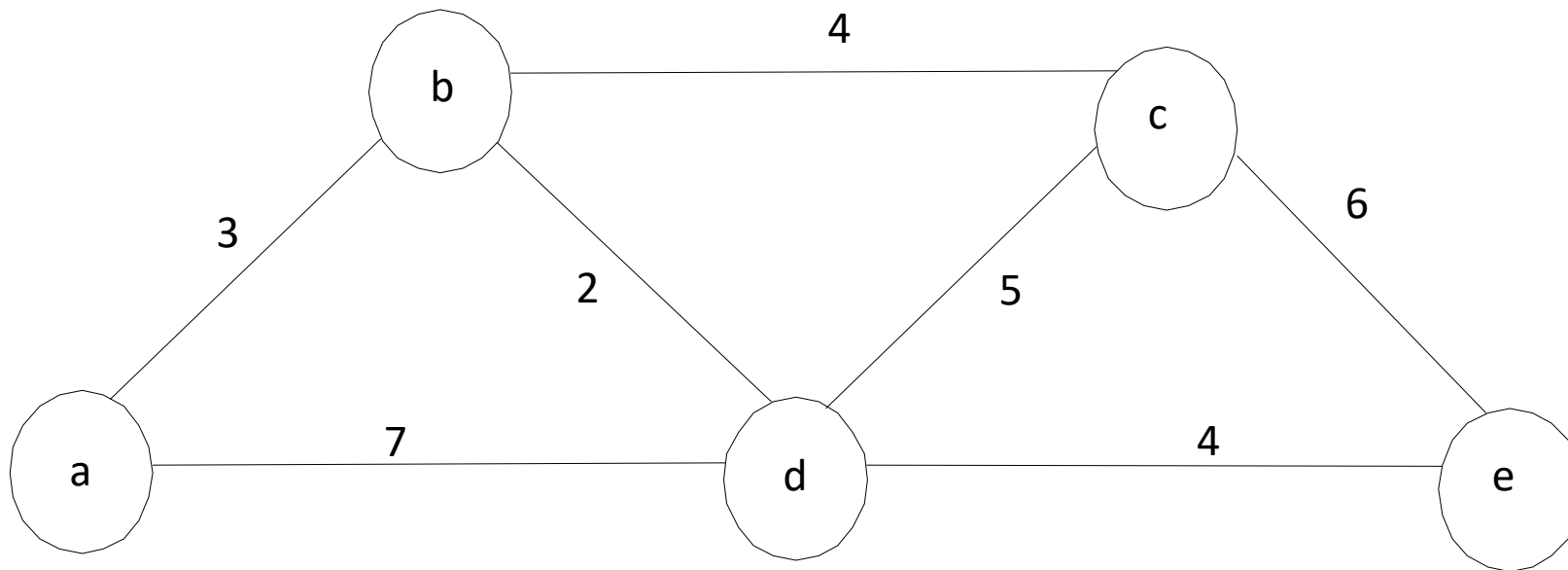
- To facilitate the algorithm's operations, we **label** each vertex with **two labels**.
- The **numeric label** d indicates the length of the shortest path from the source to this vertex found by the algorithm so far; when a vertex is added to the tree, d indicates the length of the shortest path from the source to that vertex.
- The **other label** indicates the name of the next-to-last vertex on such a path, i.e., the parent of the vertex in the tree being constructed.
- With such labeling, finding the next nearest vertex u^* becomes a simple task of finding a fringe vertex with the smallest d value. Ties can be broken arbitrarily.

Shortest paths – Dijkstra's algorithm

- After we have identified a vertex u^* to be added to the tree, we need to perform two operations:
 - Move u^* from the fringe to the set of tree vertices.
 - For each remaining fringe vertex u that is connected to u^* by an edge of weight $w(u^*, u)$ such that
 - $d_{u^*} + w(u^*, u) < d_u$,
 - update the labels of u by u^* and $d_{u^*} + w(u^*, u)$, respectively.

Example 1: Dijkstra's Algorithm

- Solve the single-source shortest-paths problem in the following graph with vertex "a" as the source:



Example 1: Dijkstra's Algorithm

Step 1

Tree vertices

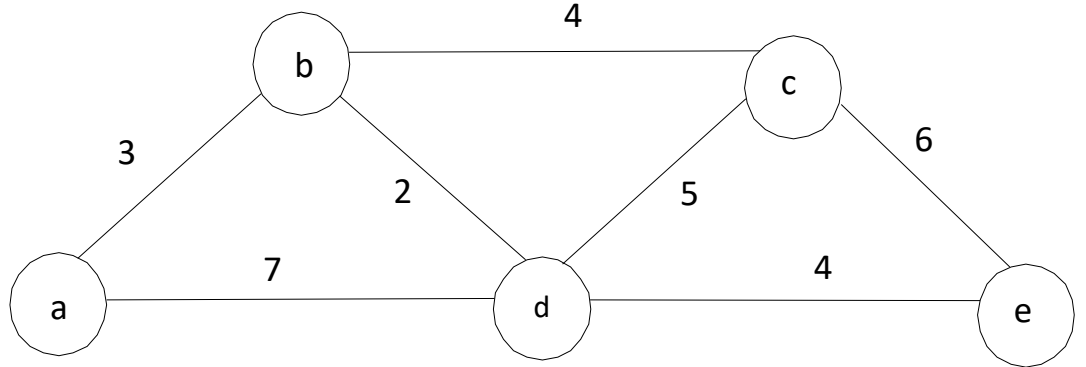
Remaining vertices

Illustration

Source Vertex

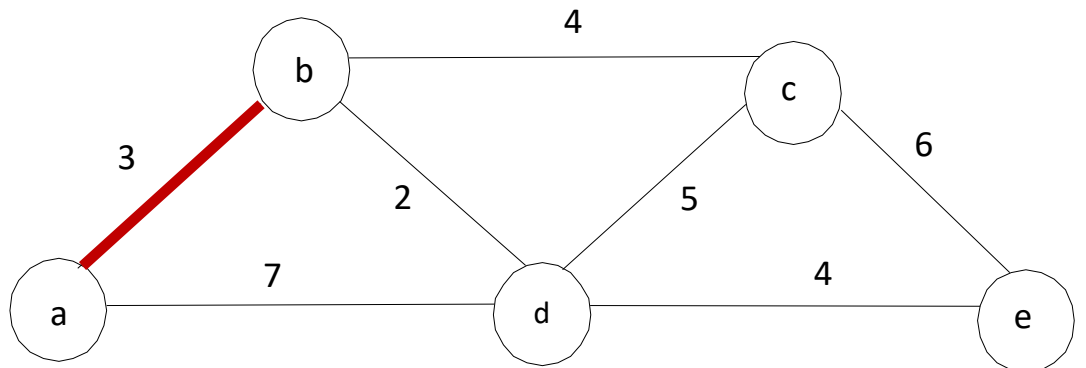
$a(-,0)$

$b(a,3)$ $c(-,\infty)$ $d(a,7)$ $e(-,\infty)$



Selected Vertex

$b(a,3)$



Example 1: Dijkstra's Algorithm

Step 2

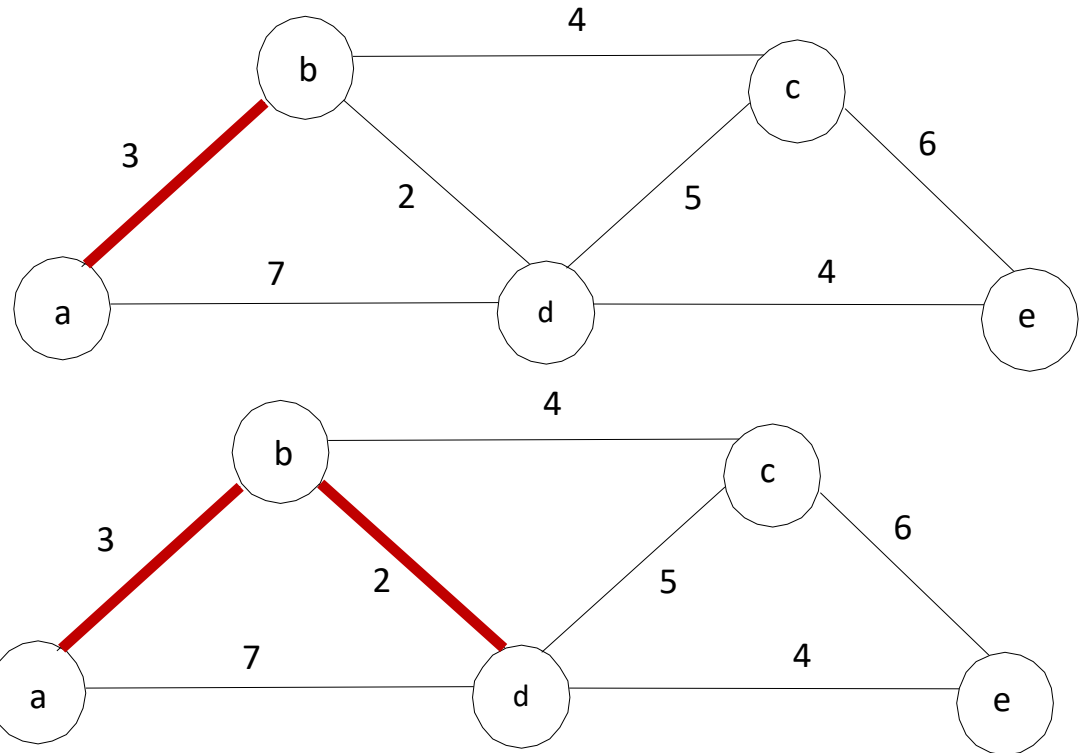
Tree vertices

Remaining vertices

Illustration

$c(b, 3+4)$ $d(b, 3+2)$ $e(-, \infty)$

Selected Vertex
 $d(b, 5)$



Step 3

Example 1: Dijkstra's Algorithm

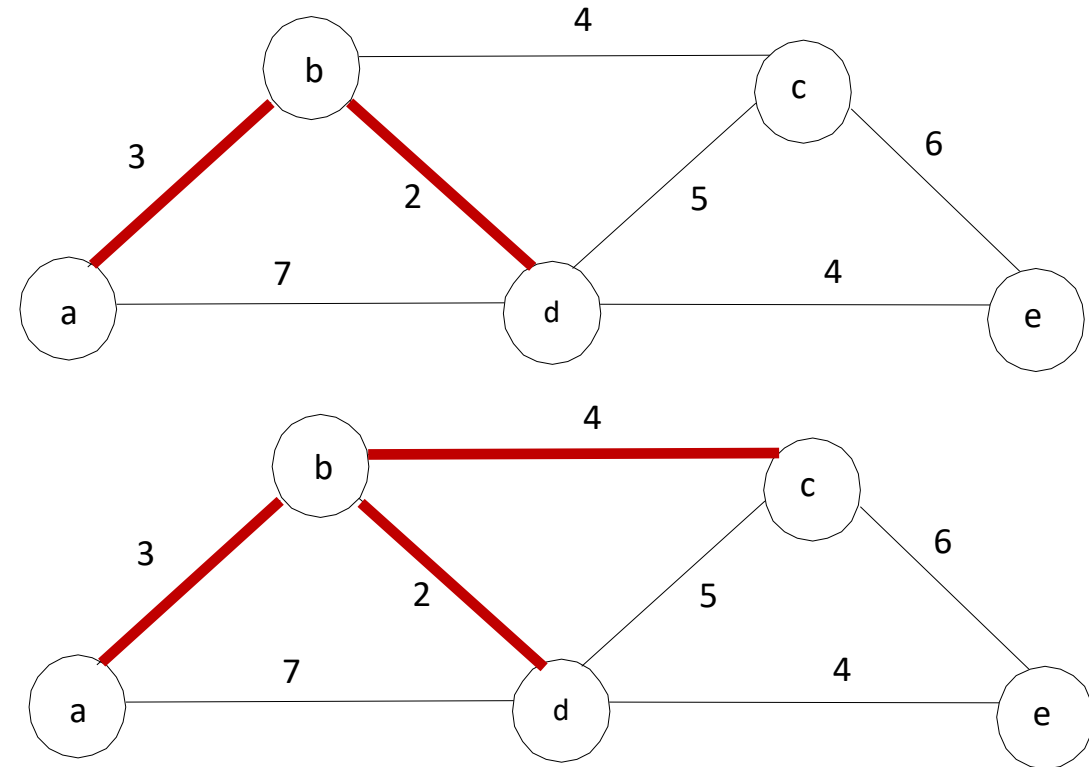
Tree vertices

Remaining vertices

Illustration

c(b,7) e(d,5+4)

Selected Vertex
c(b,7)



Example 1: Dijkstra's Algorithm

Step 4

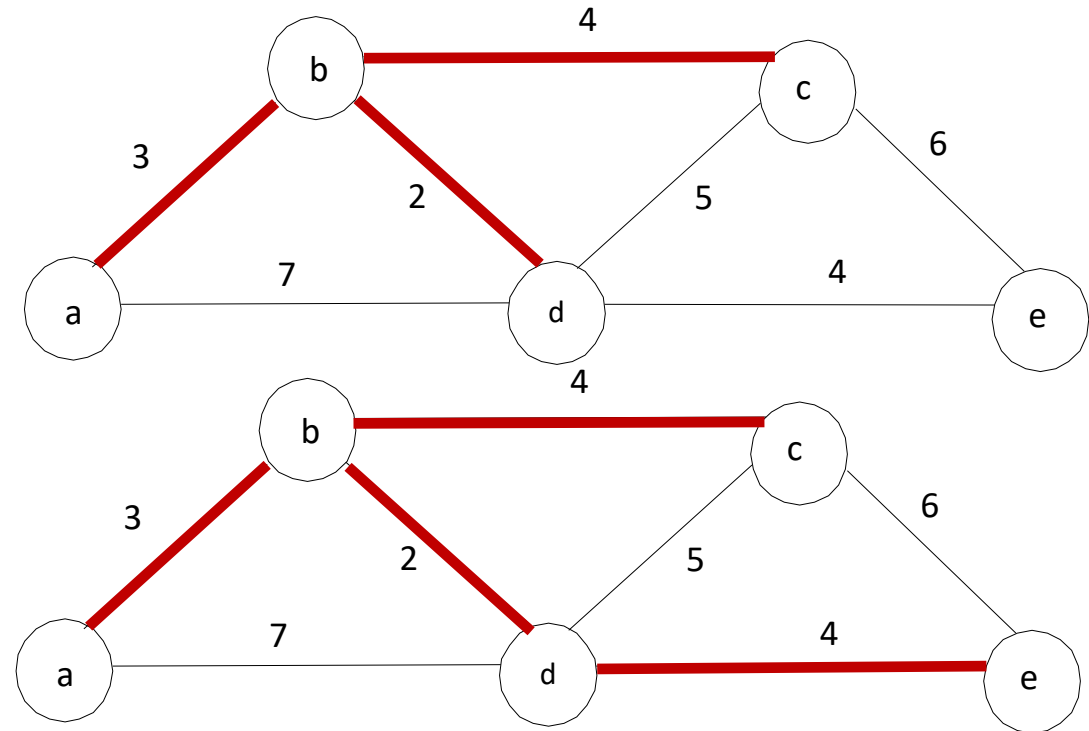
Tree vertices

Remaining vertices

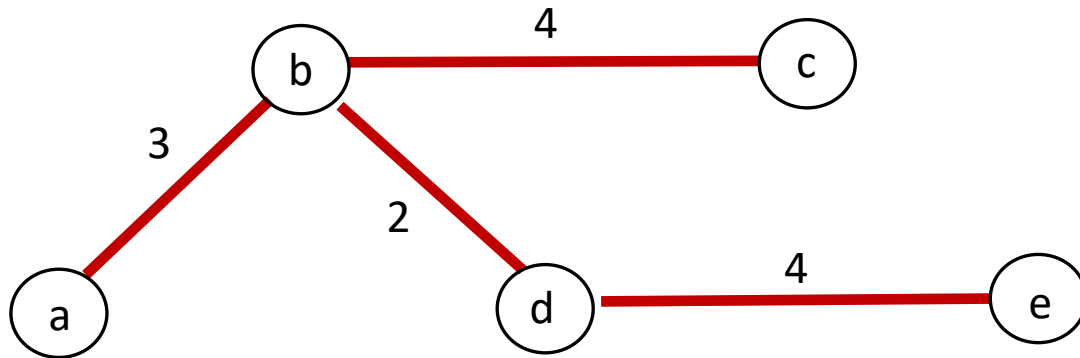
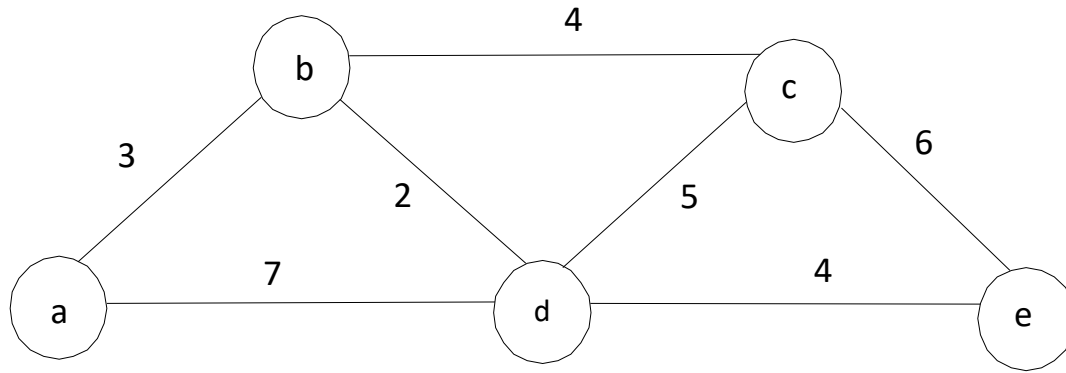
Illustration

$e(d,9)$

Selected Vertex
 $e(d,9)$



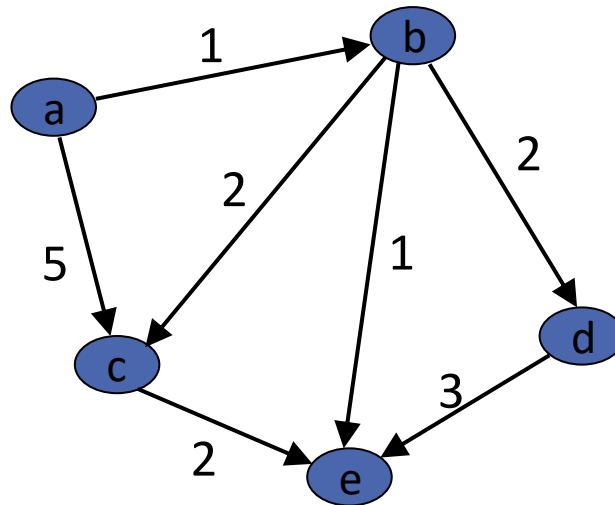
Example 1: Dijkstra's Algorithm



From a to b: a-b of Length 3
From a to d: a-b-d of Length 5
From a to c: a-b-c of Length 7
From a to e: a-b-d-e of Length 9

Example 2: Dijkstra's Algorithm-Directed Graph

- Solve the single-source shortest-paths problem in the following graph with vertex "a" as the source: :

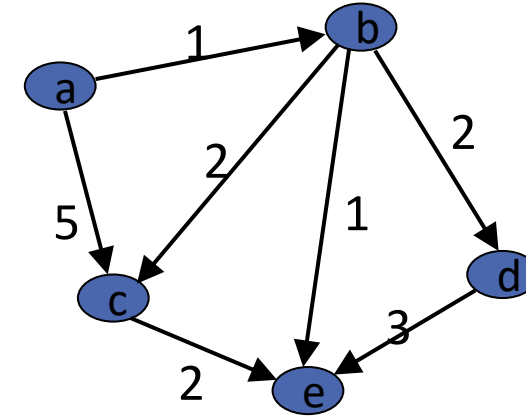


Example 2: Dijkstra's Algorithm-Directed Graph

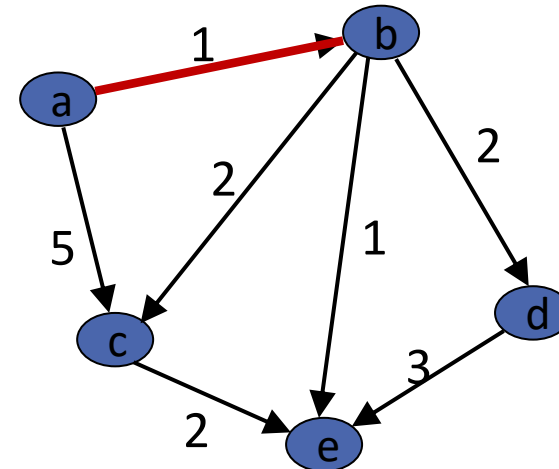
Step 1

$a(-,0)$

$b(a,1)$ $c(a,5)$ $d(-,\infty)$ $e(-,\infty)$



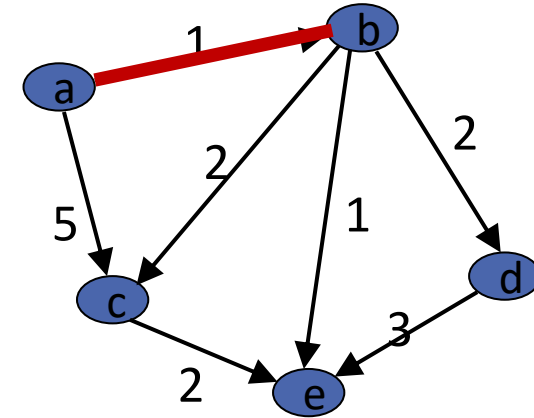
Selected Vertex
 $b(a,1)$



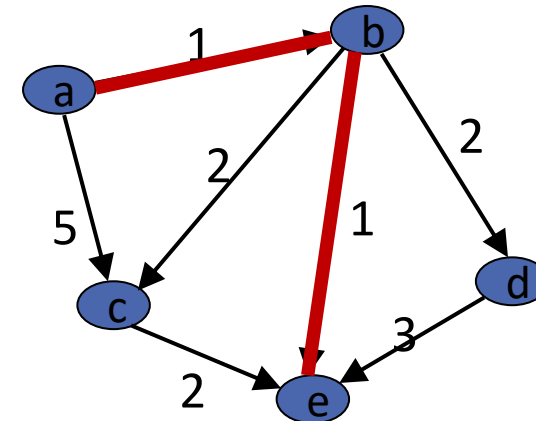
Example 2: Dijkstra's Algorithm-Directed Graph

Step 2

$c(a, 2+1)$ $d(b, 2+1)$ $e(b, 1+1)$



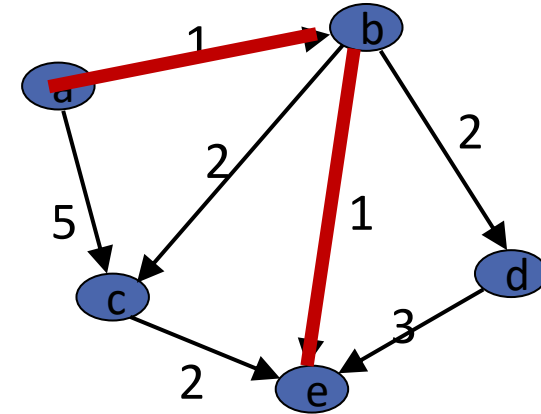
Selected Vertex
 $e(b, 2)$



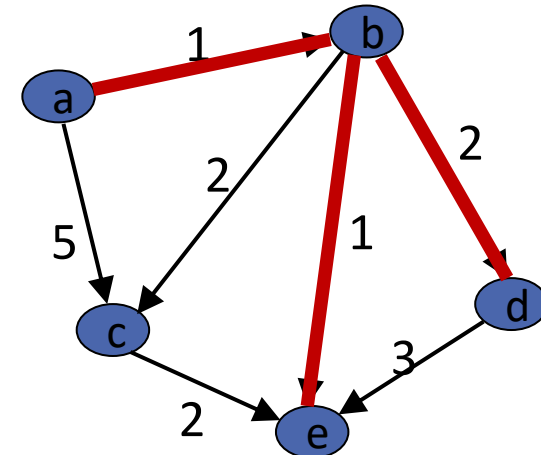
Example 2: Dijkstra's Algorithm-Directed Graph

Step 3

$c(a,3)$ $d(b,2+1)$



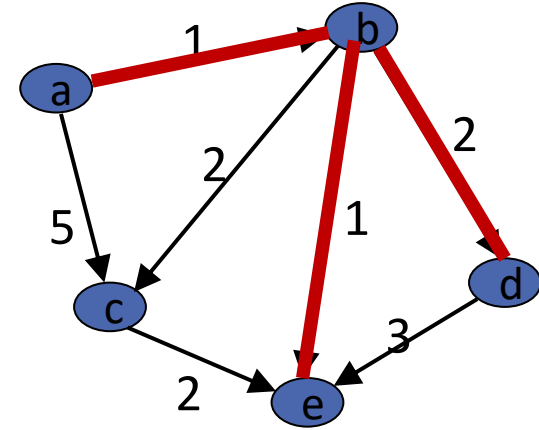
Selected Vertex
 $d(b,3)$



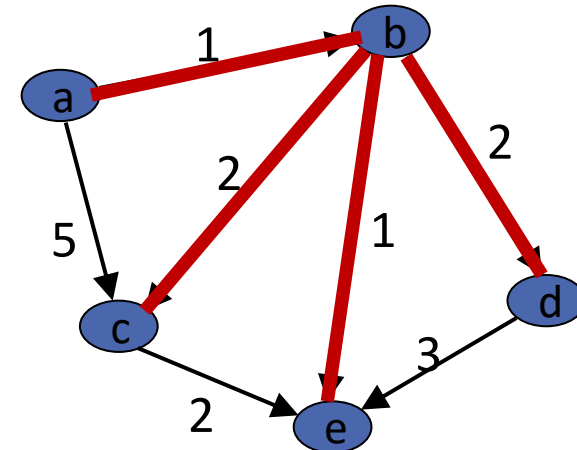
Example 2: Dijkstra's Algorithm-Directed Graph

Step 4

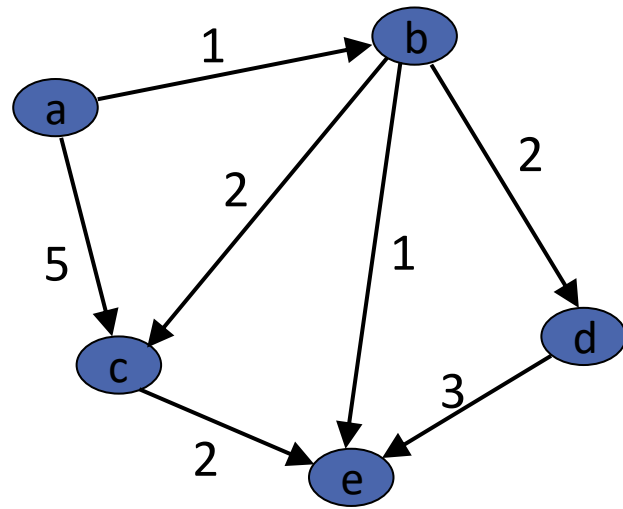
$c(b,3)$



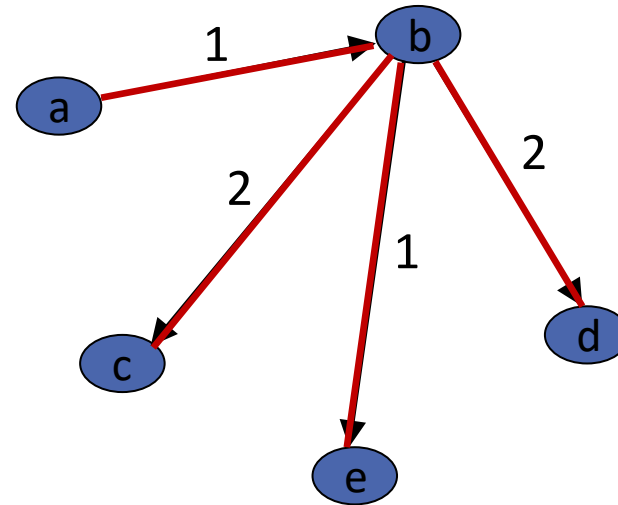
Selected Vertex
 $c(b,3)$



Example 2: Dijkstra's Algorithm-Directed Graph



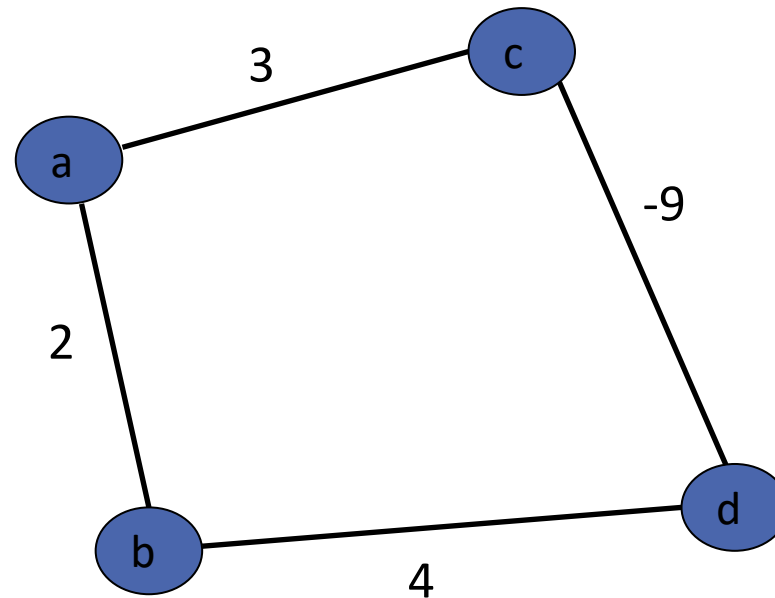
From a to b: a-b
From a to c: a-b-c
From a to e: a-b-e
From a to d: a-b-d



of Length 1
of Length 3
of Length 2
of Length 3

Example 3: Dijkstra's Algorithm-Negative Edge

- Solve the single-source shortest-paths problem in the following graph with vertex "a" as the source:

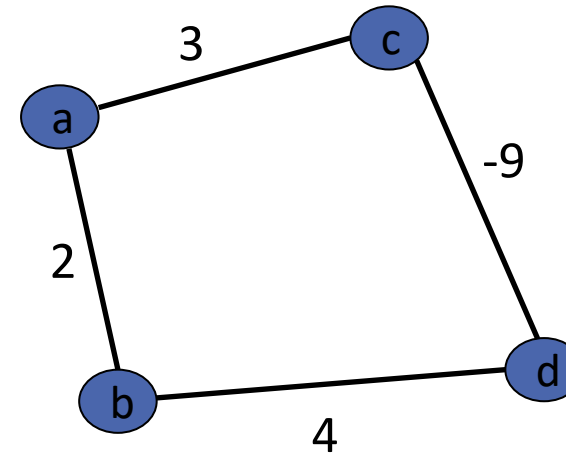


Example 3: Dijkstra's Algorithm-Negative Edge

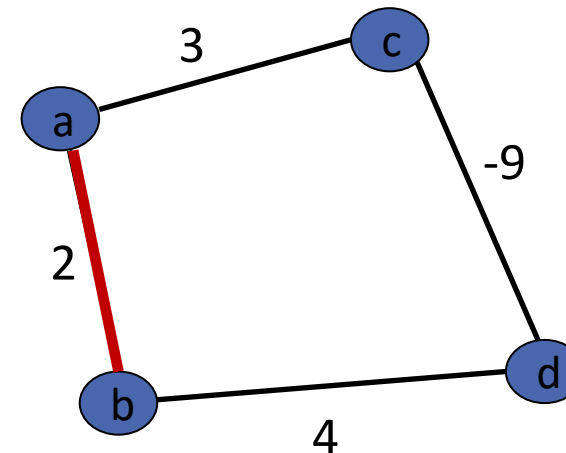
Step 1

$a(-,0)$

$b(a,2)$ $c(a,3)$ $d(-,\infty)$



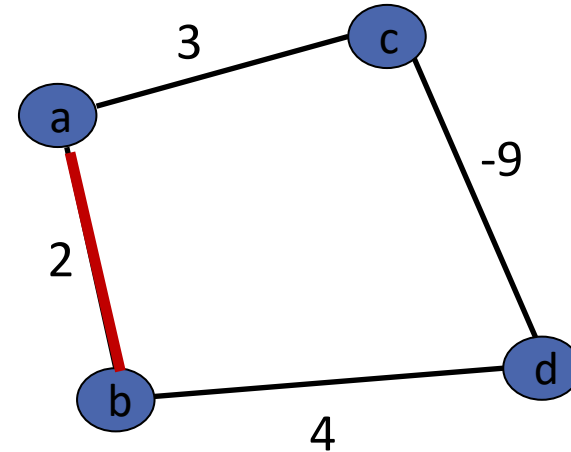
Selected Vertex
 $b(a,2)$



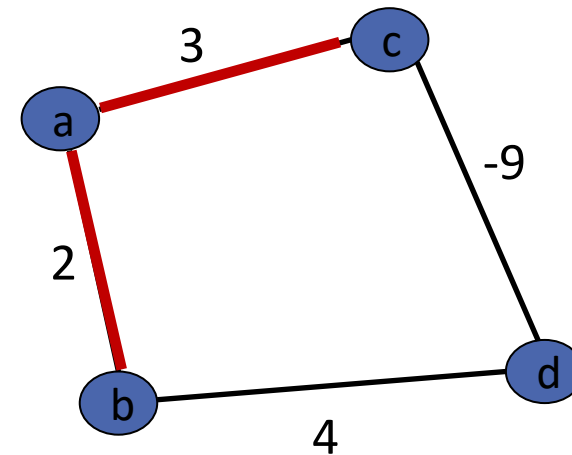
Example 3: Dijkstra's Algorithm-Negative Edge

Step 2

$c(a,3)$ $d(b,4 + 2)$



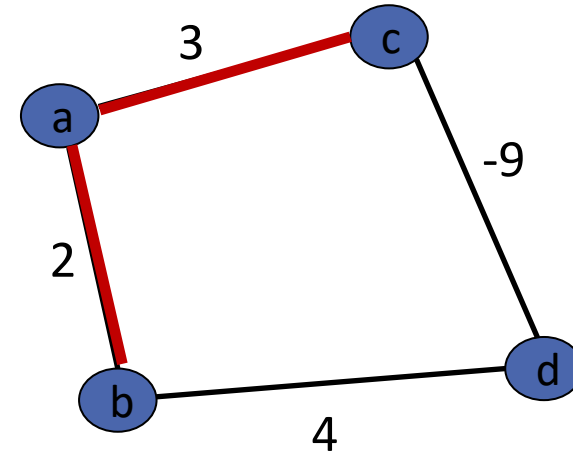
Selected Vertex
 $c(a,3)$



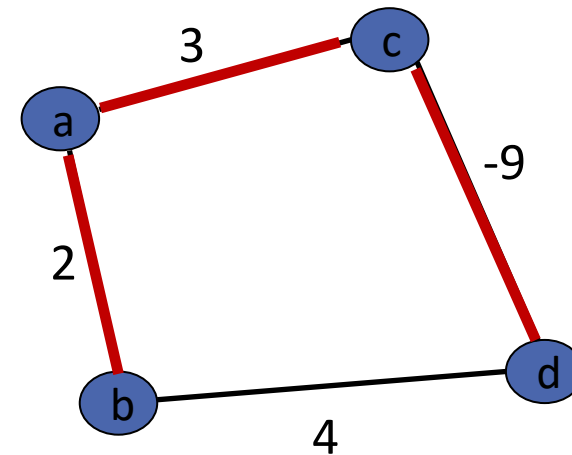
Example 3: Dijkstra's Algorithm-Negative Edge

Step 3

$d(c, -9+3)$

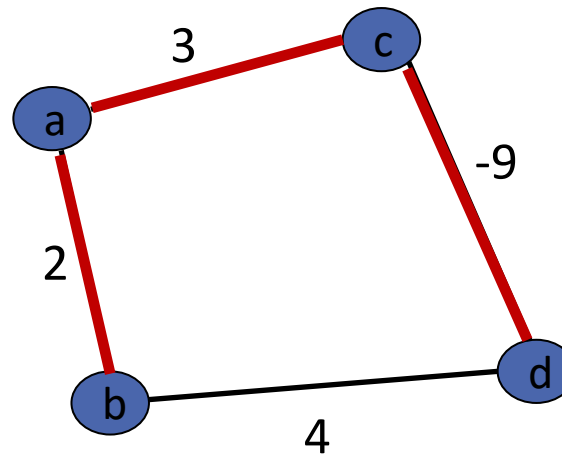


Selected Vertex
 $d(c, -6)$



Example 3: Dijkstra's Algorithm-Negative Edge

- Dijkstra's algorithm can not be applied to a graph with negative edge(s).



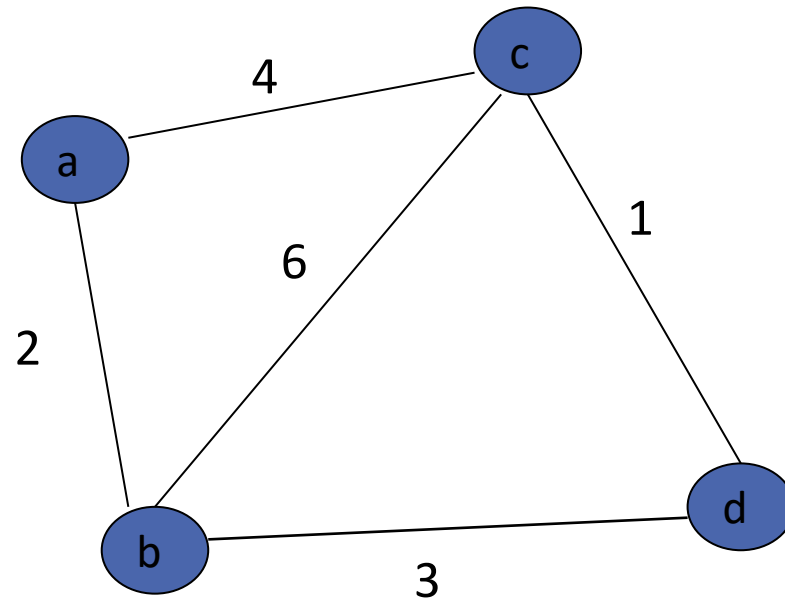
From a to b: a-b of Length 2

?

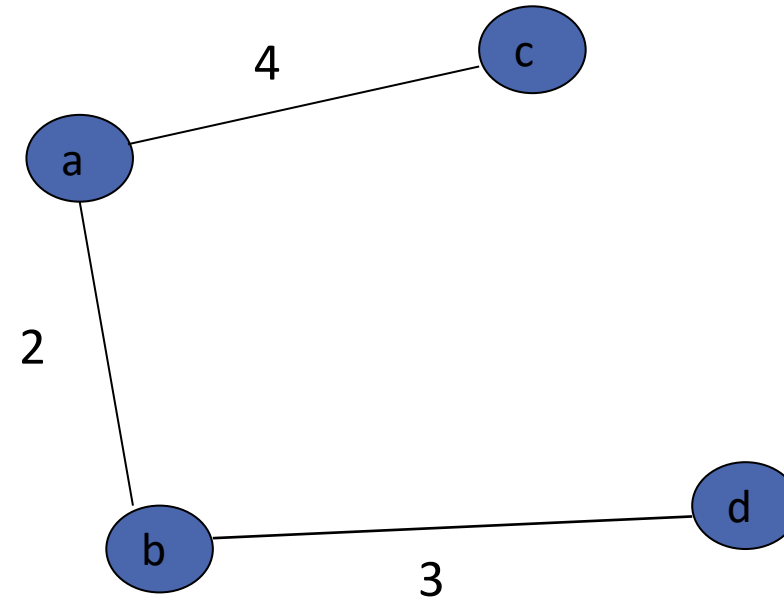
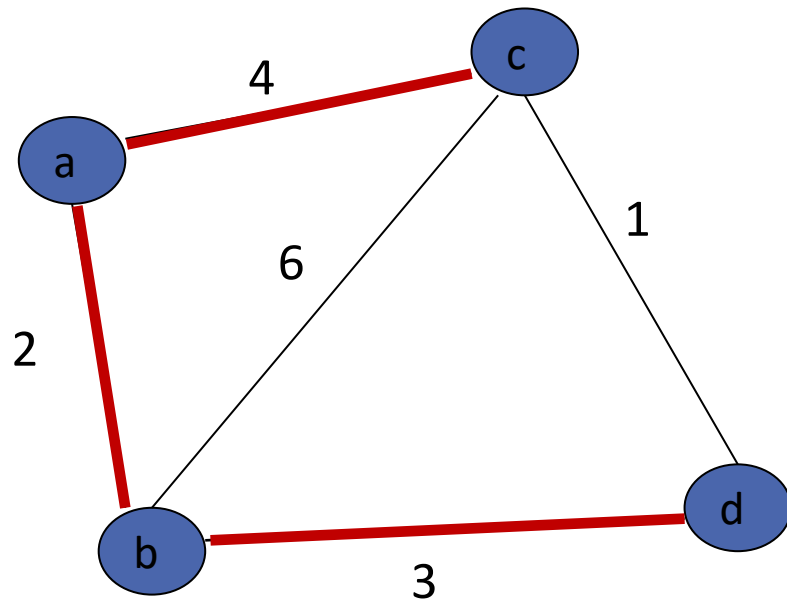
From a to b: a-c-d-b of Length -2

Example 4: Dijkstra's Algorithm

- Solve the single-source shortest-paths problem in the following graph with vertex "a" as the source:



Example 4: Dijkstra's Algorithm



Pseudocode of Dijkstra's algorithm

ALGORITHM *Dijkstra*(G, s)

//Dijkstra's algorithm for single-source shortest paths

//Input: A weighted connected graph $G = \langle V, E \rangle$ with nonnegative weights

// and its vertex s

//Output: The length d_v of a shortest path from s to v

// and its penultimate vertex p_v for every vertex v in V

Initialize(Q) //initialize priority queue to empty

for every vertex v in V

$d_v \leftarrow \infty$; $p_v \leftarrow \text{null}$

Insert(Q, v, d_v) //initialize vertex priority in the priority queue

$d_s \leftarrow 0$; *Decrease*(Q, s, d_s) //update priority of s with d_s

$V_T \leftarrow \emptyset$

for $i \leftarrow 0$ **to** $|V| - 1$ **do**

$u^* \leftarrow \text{DeleteMin}(Q)$ //delete the minimum priority element

$V_T \leftarrow V_T \cup \{u^*\}$

for every vertex u in $V - V_T$ that is adjacent to u^* **do**

if $d_{u^*} + w(u^*, u) < d_u$

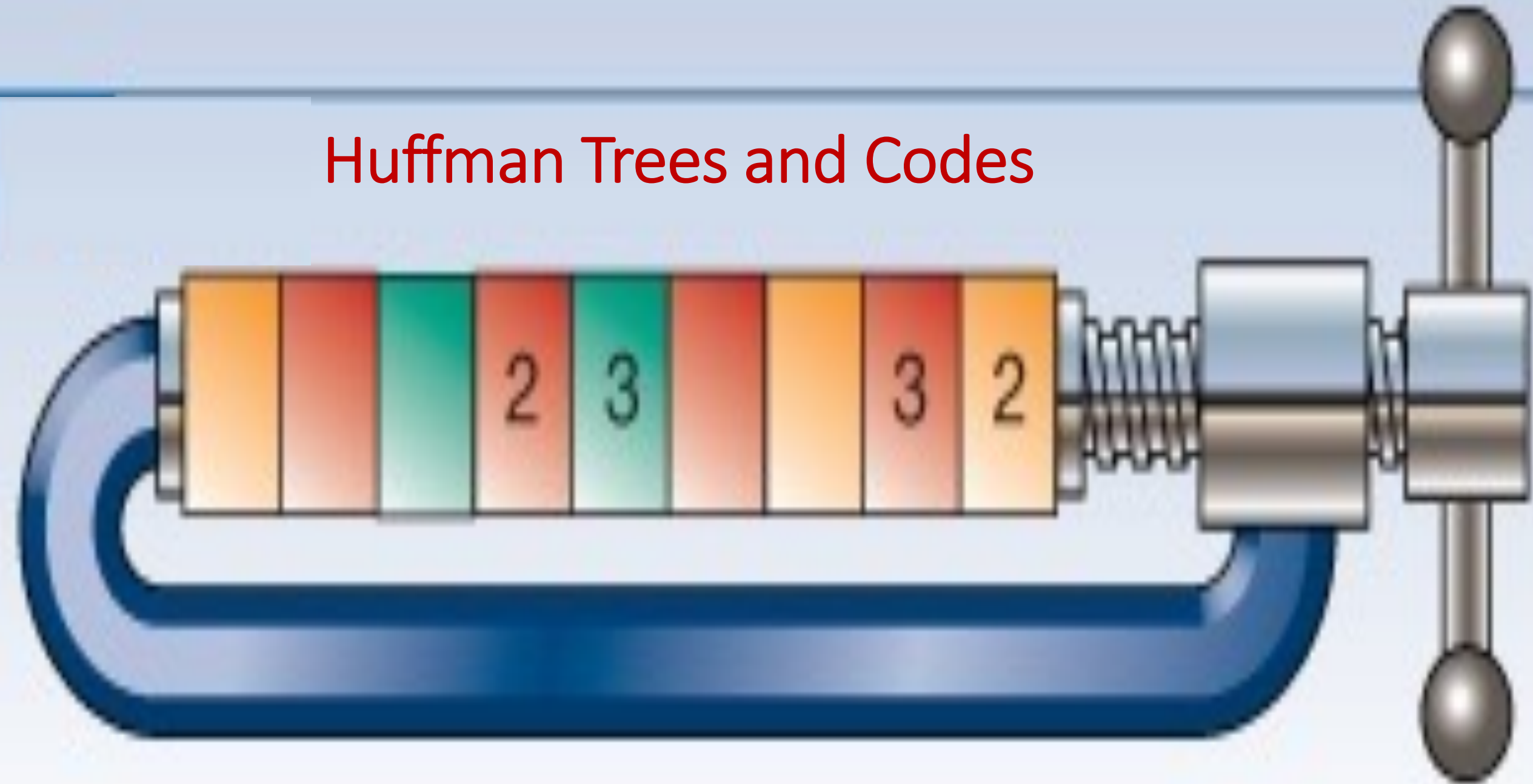
$d_u \leftarrow d_{u^*} + w(u^*, u)$; $p_u \leftarrow u^*$

Decrease(Q, u, d_u)

Analysis of Dijkstra's algorithm

- The time efficiency of Dijkstra's algorithm depends on the data structures used for **implementing the priority queue** and for representing an input graph itself.
- It is in $\Theta(|V|^2)$ for graphs represented by their weight matrix and the **priority queue implemented as an unordered array**.
- For graphs represented by their adjacency lists and the priority queue **implemented as a min-heap**, it is in $O(|E| \log |V|)$.

Huffman Trees and Codes



Coding Problem

○ Message: A B B A C C D A A B E

Characters	ASCII	8-bit	3-bit
A	65	01000001	000
B	66	01000010	001
C	67	01000011	010
D	68	01000100	011
E	69	01000101	100

○ Total for 8-bit: $10 * 8 = 80$

○ Total for 3-bit: $10 * 3 = 30$

Coding Problem

- Suppose we have to **encode** a text that comprises symbols from some n -symbol alphabet by assigning to each of the text's symbols some sequence of bits called the **codeword**.
- We can use a **fixed-length encoding** that assigns to each symbol a bit string of the same length m ($m \geq \log_2 n$).
- This is exactly what the standard ASCII code does.

Coding Problem

- One way of getting a coding scheme that yields a shorter bit string on the average is based on the old idea of **assigning shorter codewords to more frequent symbols** and **longer codewords to less frequent symbols**.
- This idea was used, in particular, in the **telegraph code** invented in the mid-19th century by Samuel Morse.
- In that code, **frequent letters** such as e (.) and a (.-) are assigned short sequences of dots and dashes while **infrequent letters** such as q (- - .-) and z (- - ..) have longer ones.
- ***Variable-length encoding***, which assigns codewords of different lengths to different symbols, introduces a problem that fixed-length encoding does not have.

Coding Problem

- If we want to create a binary prefix code for some alphabet, it is natural to associate the alphabet's symbols with leaves of a binary tree in which **all the left edges are labeled by 0** and **all the right edges are labeled by 1**.
- The **codeword** of a symbol can then be **obtained** by recording the labels on the simple path **from the root to the symbol's leaf**.
- Among the many trees that can be constructed in this manner for a given alphabet with known frequencies of the symbol occurrences, **how can we construct a tree that would assign shorter bit strings to high-frequency symbols and longer ones to low-frequency symbols?**
- It can be done by the following **greedy algorithm**, **invented by David Huffman** while he was a graduate student at MIT.

Huffman codes

- **Huffman's algorithm**
- **Step 1** : Initialize n one-node trees and label them with the symbols of the alphabet given.
- Record the frequency of each symbol in its tree's root to indicate the tree's **weight**. (More generally, the weight of a tree will be equal to the sum of the frequencies in the tree's leaves.)
- **Step 2** : Repeat the following operation until a single tree is obtained.
- Find two trees with the smallest weight.
- Make them the left and right subtree of a new tree and record the sum of their weights in the root of the new tree as its weight.

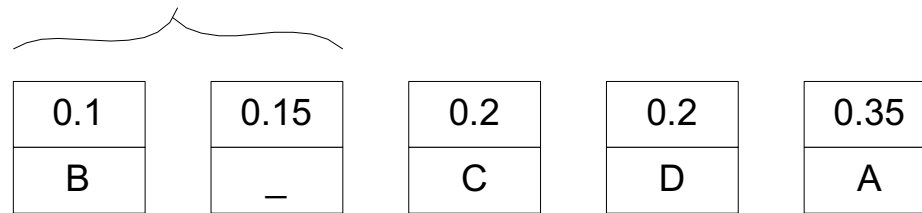
Huffman codes

- A tree constructed by the above algorithm is called a ***Huffman tree***. It defines—in the manner described above—a ***Huffman code***.
- A ***Huffman tree*** is a **binary tree** that minimizes the weighted path length from the root to the leaves of predefined weights.
- The most important application of Huffman trees is **Huffman codes**.
- A ***Huffman code*** is an optimal prefix-free variable-length encoding scheme that assigns bit strings to symbols based on their frequencies in a given text.
- This is accomplished by a greedy construction of a binary tree whose leaves represent the alphabet symbols and whose edges are labeled with 0's and 1's.

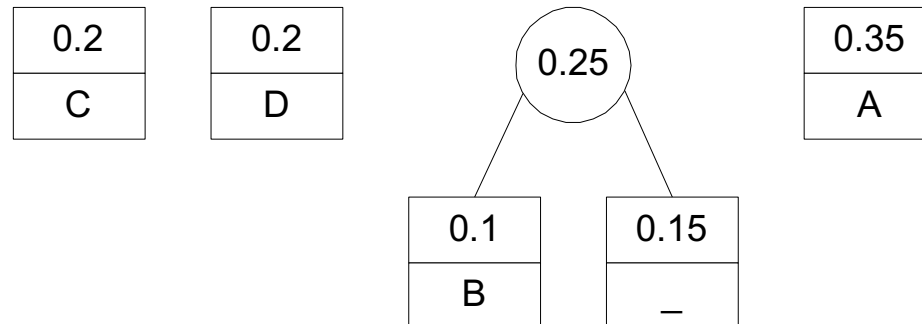
Example 1: Huffman Coding

Step 1

character	A	B	C	D	_
frequency	0.35	0.1	0.2	0.2	0.15



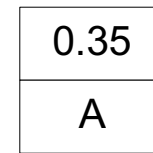
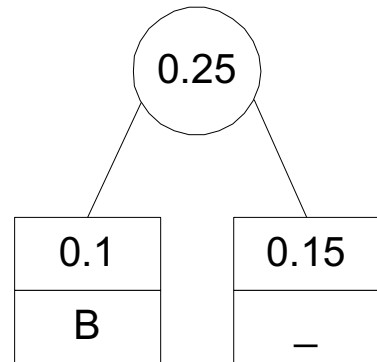
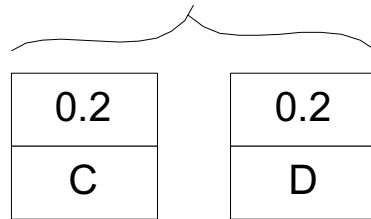
- 1- Order according to the frequencies
- 2- Select minimum two frequencies and create a tree



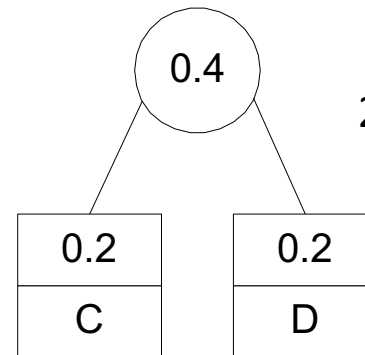
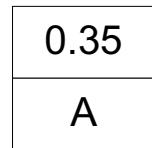
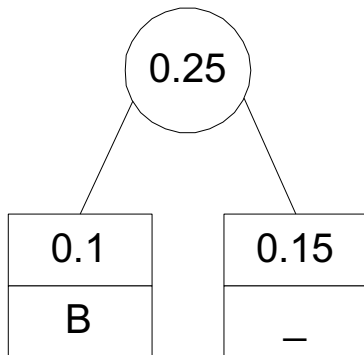
- 3- Reorder according to the frequencies

Example 1: Huffman Coding

Step 2



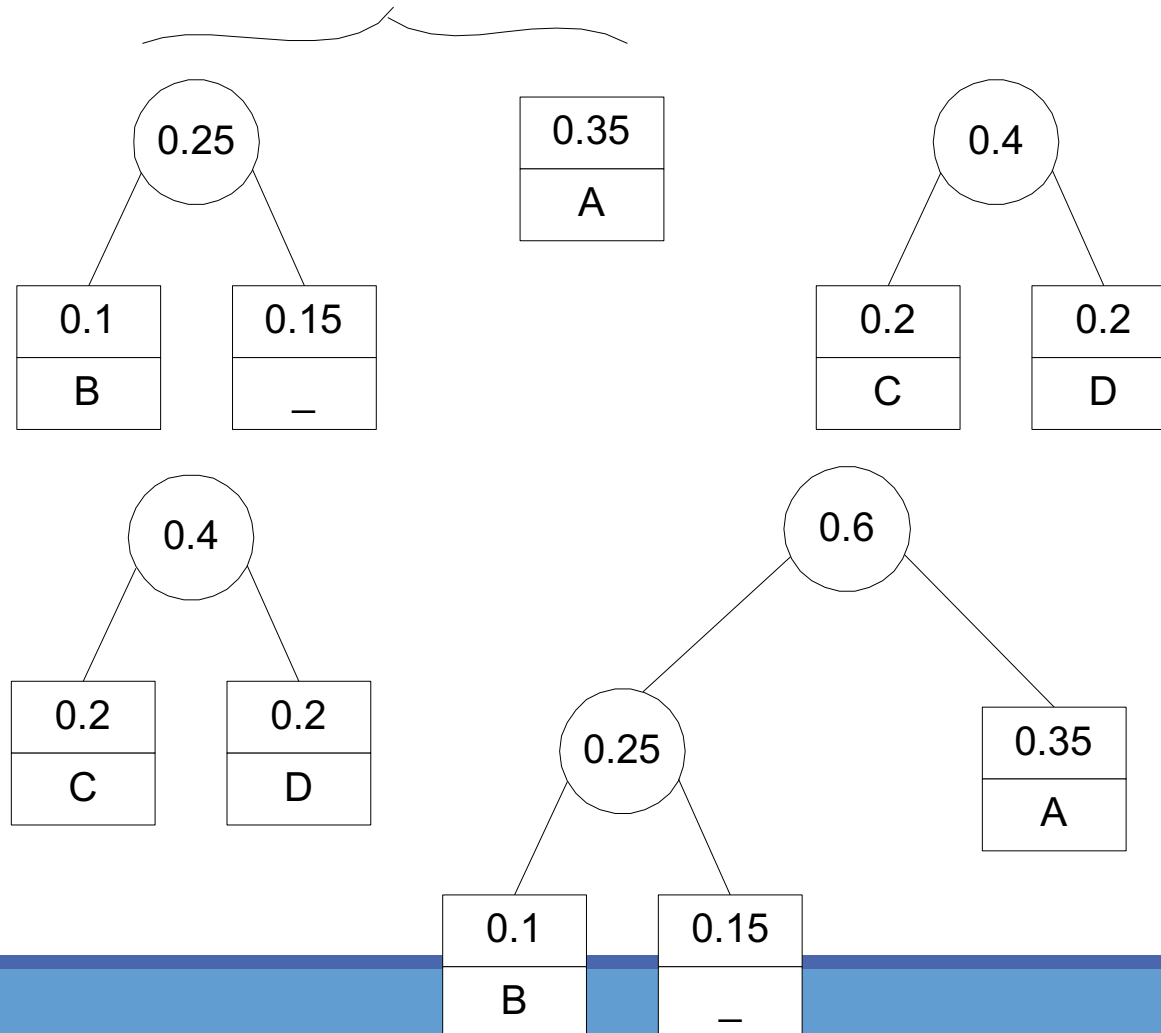
1- Select minimum two frequencies and create a tree



2- Reorder according to the frequencies

Example 1: Huffman Coding

Step 3

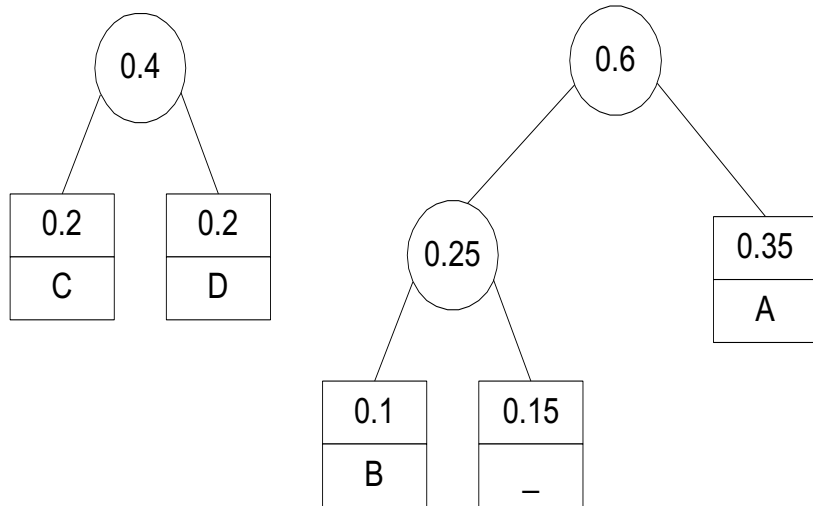


1- Select minimum two frequencies and create a tree

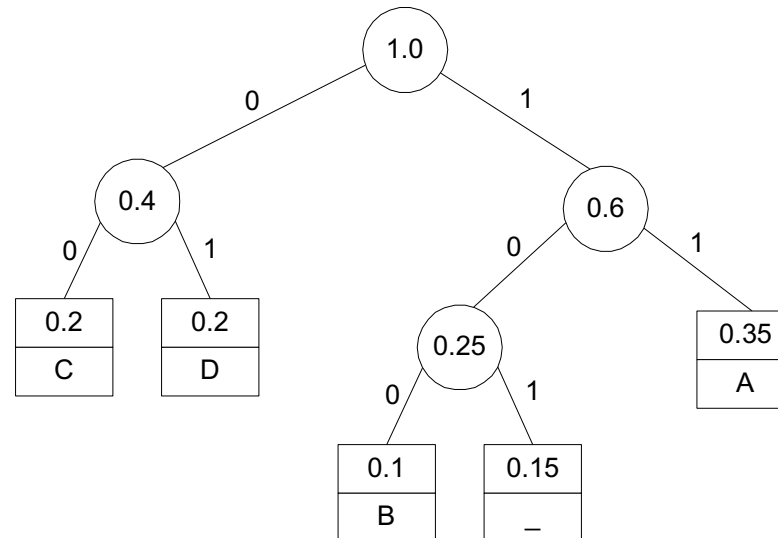
2- Reorder according to the frequencies

Example 1: Huffman Coding

Step 4



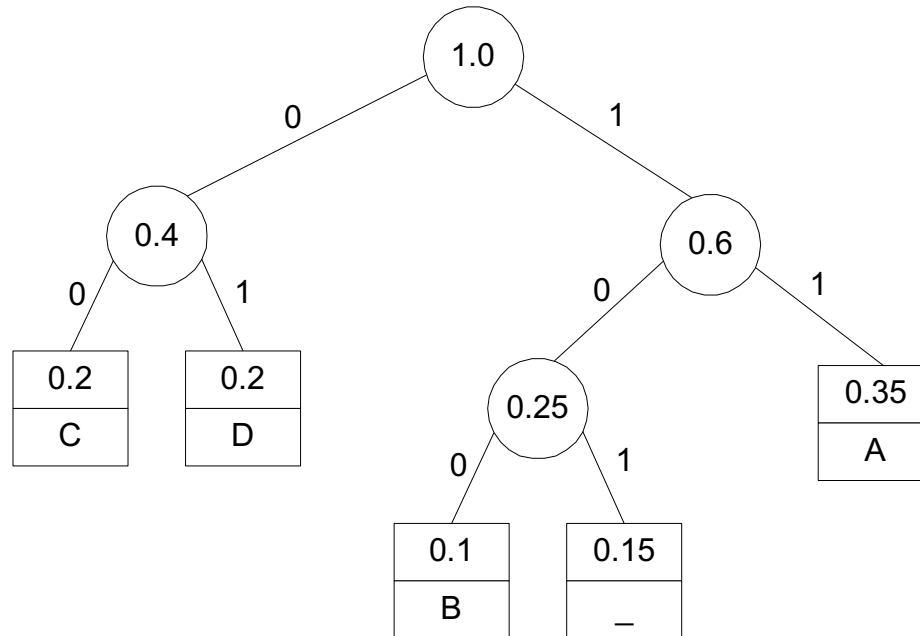
1- Select minimum two frequencies and create a tree



2- Reorder according to the frequencies
3- If root is one, It is done.

Example 1: Huffman Coding

Step 5



All the left edges are labeled by 0
All the right edges are labeled by 1.

The resulting codewords

symbol	A	B	C	D	-
frequency	0.35	0.1	0.2	0.2	0.15
codeword	11	100	00	01	101

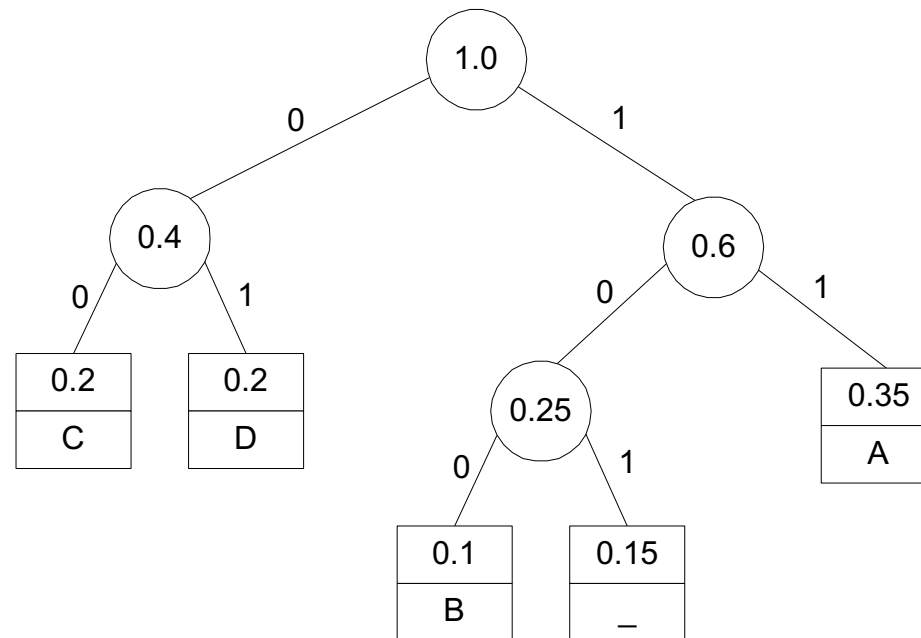
Example 1: Huffman Coding

character	A	B	C	D	_
codeword	11	100	00	01	101

- With the occurrence frequencies given and the codeword lengths obtained, the average number of bits per symbol in this code is
$$2 * 0.35 + 3 * 0.1 + 2 * 0.2 + 2 * 0.2 + 3 * 0.15 = 2.25$$
- Had we used a fixed-length encoding for the same alphabet, we would have to use at least **3 bits per each symbol**.
- For this example, Huffman's code achieves the **compression ratio**—a standard measure of a compression algorithm's effectiveness—of
$$(3 - 2.25)/3 \cdot 100\% = 25\%.$$
- In other words, Huffman's encoding of the text will use 25% less memory than its fixed-length encoding.

Example 1: Huffman Coding

- DAD is encoded as 011101
- 10011011011101 is decoded as BAD_AD.



Example 2: Huffman Coding

- Construct a Huffman code for the following message

Message: A B B A C C D A A B

Example 2: Huffman Coding

Step 1

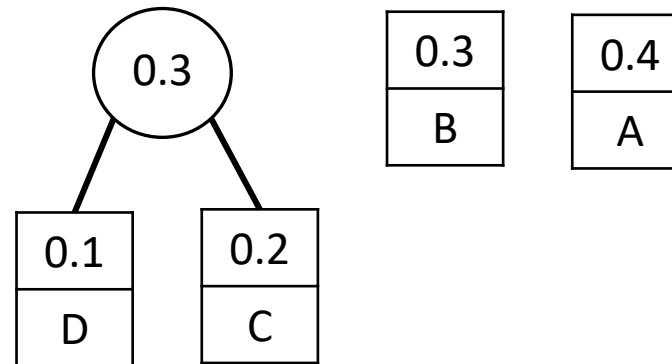
Message: A B B A C C D A A B

character	A	B	C	D
frequency	4/10	3/10	2/10	1/10

0.1	0.2	0.3	0.4
D	C	B	A

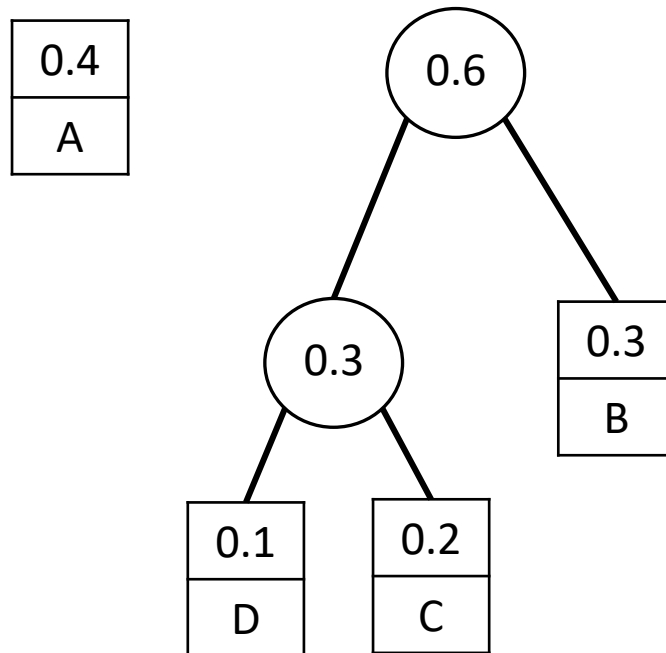
Example 2: Huffman Coding

Step 2



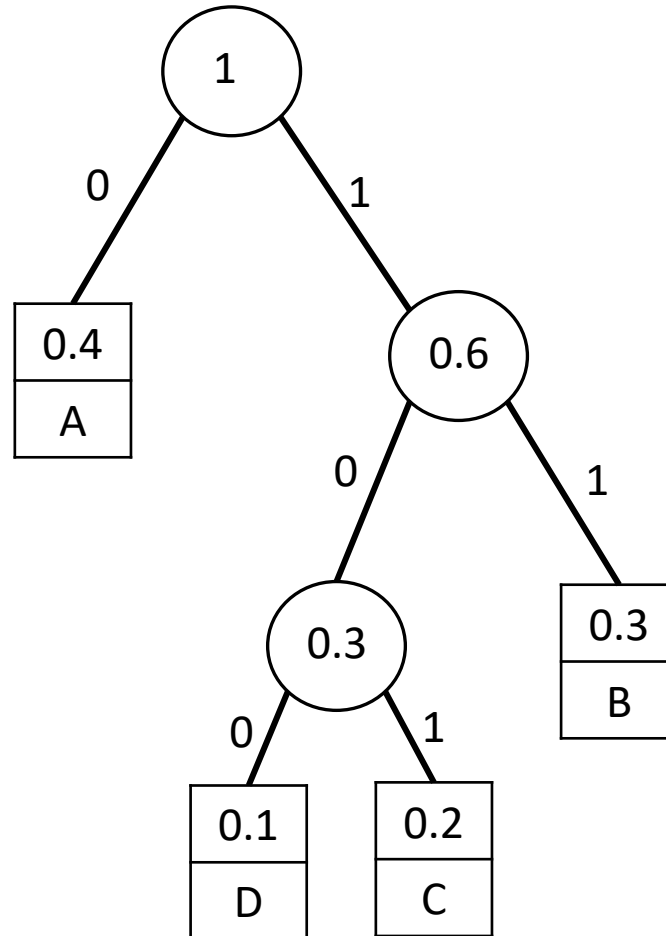
Example 2: Huffman Coding

Step 3



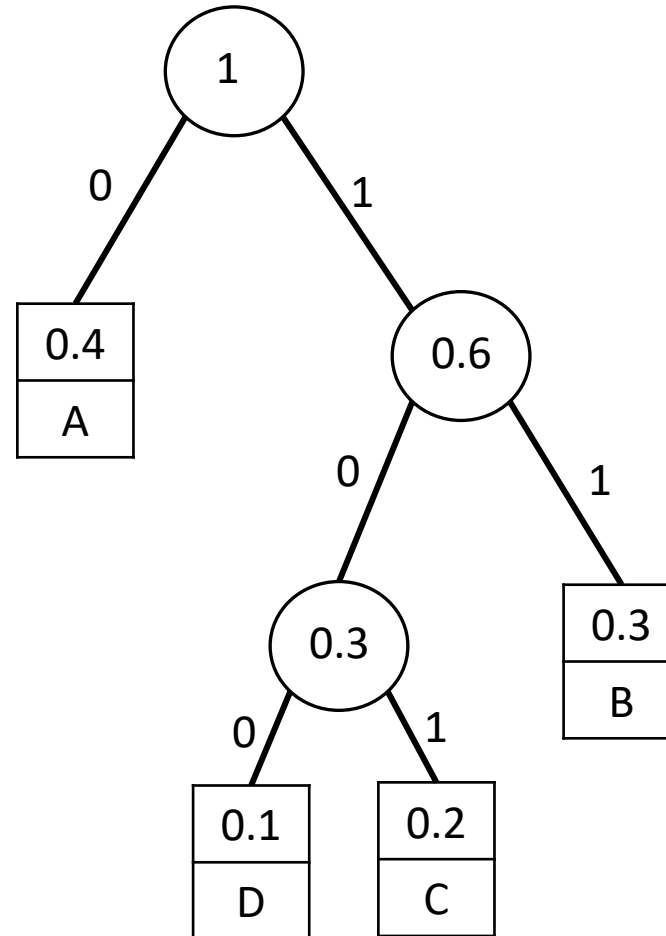
Example 2: Huffman Coding

Step 4



Example 2: Huffman Coding

Step 5



A → 0
B → 11
C → 101
D → 100

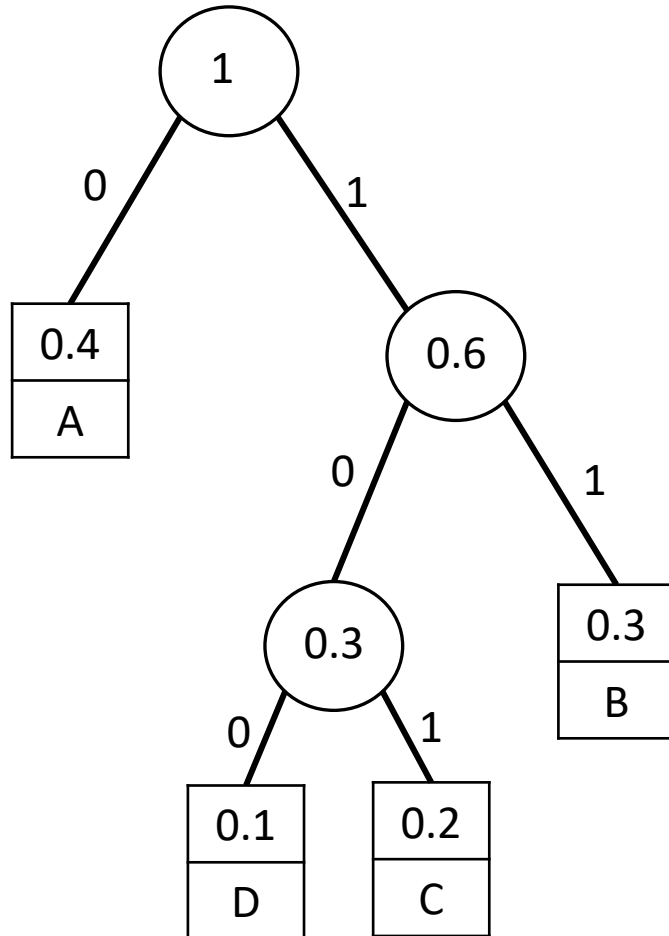
Example 2: Encoding

A → 0
B → 11
C → 101
D → 100

Message (10 Characters): A B B A C C D A A B

Codeword (19 bits). : 0111101011011000011

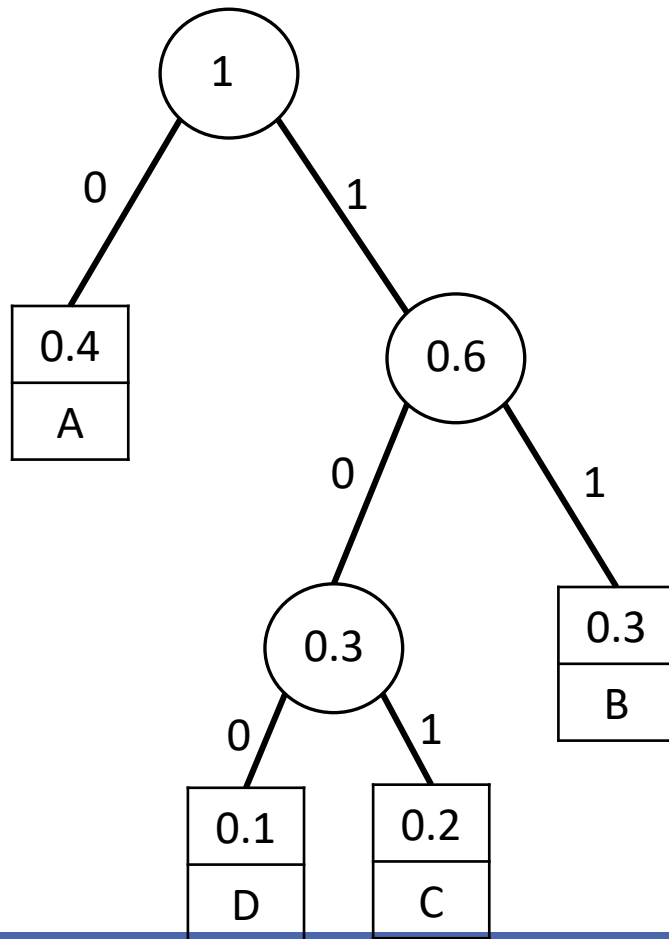
Example 2: Decoding



Codeword : 011011110101101100

Message : ?

Example 2: Decoding



Codeword : 011011110101101100

Message : A B A B B A C C D

Summary

- Greedy Technique
- Prim's Algorithm
- Single-source shortest-paths
- Dijkstra's Algorithm
- Huffman Coding

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.